



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1

System Integration and Architecture

System Design Document (SDD)

Project Title: WashQ – Online Laundry Reservation System

Prepared By: Abel, Zydric Quimbo

Version: 1.0

Date: February 28, 2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	Feb 21, 2026	Abel, Zydric Q.	Initial draft created	Final
1.0	Feb 28, 2026	Abel, Zydric Q.	Baseline version for development	Final

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS.....	3
Contents.....	3
EXECUTIVE SUMMARY.....	3
1.0 INTRODUCTION.....	6
1.1 Purpose.....	6
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION.....	6
2.1 Project Overview.....	6
2.2 Core User Journeys.....	6
2.3 Feature List (MoSCoW).....	7
2.4 Detailed Feature Specifications.....	8
2.5 Acceptance Criteria.....	11
3.0 NON-FUNCTIONAL REQUIREMENTS.....	13
3.1 Performance Requirements.....	13
3.2 Security Requirements.....	13
3.3 Compatibility Requirements.....	14
3.4 Usability Requirements.....	14
4.0 SYSTEM ARCHITECTURE.....	15
4.1 Component Diagram.....	15
4.2 Technology Stack.....	16
4.3 Backend Module Structure.....	17
5.0 API CONTRACT & COMMUNICATION.....	18
5.1 API Standards.....	18
5.2 Endpoint Specifications.....	18
5.3 Error Handling.....	25
6.0 DATABASE DESIGN.....	28
6.1 Entity Relationship Diagram.....	28
6.2 Entity Relationships.....	28
6.3 Database Tables.....	29
6.4 Database Design Notes.....	32
7.0 UI/UX DESIGN.....	33
7.1 Web Application Design System.....	33
7.2 Web Application Screens.....	35
7.3 Web Screens Summary.....	58
7.4 Mobile Application Design System.....	59
7.5 Mobile Application Screens.....	64
7.6 Mobile Screens Summary.....	91
8.0 PLAN.....	92
8.1 Project Timeline.....	92
8.2 Milestones.....	94
8.3 Risk Mitigation.....	95

EXECUTIVE SUMMARY

1.1 Project Overview

WashQ is an online laundry reservation system designed for a single laundry shop serving university students, local residents, and boarding house tenants. The system enables customers to book laundry service slots online, track their order status in real time, and receive email notifications when their laundry is ready for pickup. Shop staff and administrators are provided with a dedicated dashboard to manage bookings, update order progress, and manage available services.

WashQ is delivered as a three-platform system: a Spring Boot REST API backend, a React web application, and a native Android mobile application. All three platforms share the same backend and database, ensuring consistency across channels.

1.2 Objectives

1. Develop a fully functional laundry reservation system with real-time slot availability and double-booking prevention.
2. Implement a three-tier architecture using Spring Boot (backend), React (web frontend), and Android Kotlin (mobile).
3. Integrate Google OAuth 2.0 for social login alongside traditional email/password registration.
4. Integrate PayMongo as the payment gateway in sandbox mode to support online payment processing.
5. Implement SMTP-based email notifications for account-related and order-related events.
6. Consume the OpenWeatherMap public API to display a contextual weather widget on the customer dashboard.
7. Support file upload functionality for customer profile pictures stored on the server.
8. Enforce role-based access control at both the API level and UI level for Customer and Staff/Admin roles.

9. Deliver a native Android application using Kotlin and XML-based UI layouts integrated with the same backend.

1.3 Scope

Included Features:

- User registration with email/password and Google OAuth 2.0 login
- JWT-based authentication with BCrypt password hashing
- Role-based access control for Customer and Staff/Admin roles
- Service catalog browsing (Wash Only, Wash & Dry, Wash/Dry/Fold, Dry Clean, Iron Only)
- Time slot availability viewing and selection
- Booking creation with real-time slot locking to prevent double-booking
- Order status tracking for customers (Pending, Received, In Progress, Ready for Pickup, Completed, Cancelled)
- Staff/Admin dashboard for managing bookings and updating order status
- PayMongo sandbox payment processing linked to each booking
- SMTP email notifications: welcome email on registration and booking/order-ready notifications
- OpenWeatherMap API integration with a weather widget on the customer dashboard
- Profile picture upload (JPEG/PNG) stored on the server and linked to the user account
- Protected routes on both the web and mobile applications
- Responsive React web application consuming the Spring Boot REST API
- Android mobile application (Kotlin, XML layouts, API Level 34) integrated with the backend

Excluded Features:

- Multi-shop or franchise support
- Real (live) payment processing — sandbox only
- SMS or push notification support
- Product inventory management automation
- Advanced analytics or reporting dashboard
- Multi-language or localization support
- Proof-of-completion photo upload by staff (possible future enhancement)
- Customer reviews and ratings

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for WashQ, the Online Laundry Reservation System. It provides detailed functional and non-functional requirements, architectural decisions, API contracts, database schema design, and an implementation roadmap to guide development across all three platforms: the Spring Boot backend, the React web application, and the Android mobile application.

This document is intended for the developer (Abel, Zydric Quimbo) and the course instructor for IT342 – System Integration and Architecture. It is a living document that will be updated as development progresses.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

1. **Project Name:** WashQ – Online Laundry Reservation System
2. **Domain:** Service Reservation / Laundry
3. **Primary Users:**
 1. Customers (students, residents, tenants)
 2. Staff / Administrators
4. **Problem Statement:** Customers of walk-in laundry shops have no way to reserve a service slot in advance, leading to long queues, uncertainty about order status, and missed pickup times. Shop staff manage all bookings manually with no centralized system.
5. **Solution:** WashQ provides an online reservation platform where customers can book a laundry slot, pay online, and track their order status. Staff and admins manage the shop workflow through a dedicated dashboard with real-time order updates.

2.2 Core User Journeys

- **Journey 1: Customer Books a Laundry Slot (Happy Path)**
 1. Customer visits the WashQ web or mobile application.
 2. Customer registers an account or logs in via email/password or Google OAuth.
 3. Customer receives a welcome email upon successful registration.
 4. Customer views the dashboard displaying available services, time slots, and the weather widget.

5. Customer selects a laundry service (e.g., Wash & Dry) and an available time slot.
 6. Customer confirms the booking. The system checks slot availability, locks the slot, and creates the order.
 7. Customer is redirected to the payment page and completes payment via PayMongo.
 8. Customer receives a booking confirmation email with order details.
 9. Customer monitors order status on the dashboard (Pending → Received → In Progress → Ready for Pickup).
 10. Customer receives an email notification when laundry is ready for pickup.
 11. Customer picks up laundry and the status is updated to Completed.
- **Journey 2: Staff/Admin Manages Bookings**
 1. Staff logs in with staff credentials.
 2. Staff views the dashboard showing all current and upcoming bookings.
 3. Staff updates a booking status as the order progresses through the workflow.
 4. Upon marking an order as Ready for Pickup, the system automatically sends an email notification to the customer.
 5. Staff can view booking details including service type, customer information, and payment status.
 6. Admin can add, edit, or deactivate laundry services and manage time slot availability.
 - **Journey 3: Customer Updates Profile**
 1. Customer logs in and navigates to the Profile page.
 2. Customer views current profile information (name, address, contact number, profile picture).
 3. Customer uploads a new profile picture (JPEG/PNG).
 4. Customer edits name, address, or contact number and saves changes.
 5. System updates the record and displays the updated profile.

2.3 Feature List (MoSCoW)

MUST HAVE

1. User registration (email/password) with welcome email
2. Google OAuth 2.0 login with JWT issuance
3. JWT-based session management with BCrypt password hashing
4. Role-Based Access Control: Customer and Staff/Admin
5. Service catalog with full CRUD (admin-managed)
6. Time slot viewing and availability checking
7. Booking creation with slot locking and double-booking prevention
8. Order status tracking (Customer view)

9. Staff dashboard for booking management and status updates
10. PayMongo sandbox payment integration
11. SMTP email: welcome email and order-ready notification
12. OpenWeatherMap weather widget on customer dashboard
13. Profile picture upload (JPEG/PNG)
14. Protected routes on web and mobile
15. Android mobile app (Kotlin, XML, API 34, Retrofit)

SHOULD HAVE

1. Customer booking cancellation
2. Order history list for customers
3. Proper input validation with descriptive error messages
4. Responsive web design across mobile, tablet, and desktop viewports
5. Booking confirmation email with order summary

COULD HAVE

1. Real-time order status updates via WebSocket or polling (bonus feature)
2. Proof-of-completion photo upload by staff
3. Special instructions field on booking form
4. Admin ability to configure time slot capacity

WON'T HAVE

1. Multi-shop support
2. Live (non-sandbox) payment processing
3. SMS notifications
4. Mobile push notifications
5. Inventory management automation
6. Advanced analytics or reporting
7. Multi-language support

2.4 Detailed Feature Specifications

Feature: User Authentication

- **Screens:** Registration, Login, Google OAuth Redirect
- **Fields:** First Name, Last Name, Email, Password, Address, Contact Number
- **Validation:** Email format, password minimum 8 characters, unique email, required fields
- **API Endpoints:**
 - POST /auth/register,

- o POST /auth/login,
 - o POST /auth/logout,
 - o GET /auth/me,
 - o POST /auth/google
- **Security:** JWT tokens, BCrypt password hashing (salt rounds = 12), Google OAuth 2.0
- **Email Trigger:** Welcome email on successful registration

Feature: Role-Based Access Control

- **Roles:** CUSTOMER, STAFF, ADMIN
- **Customer Access:** Book services, view own bookings, track order status, manage own profile
- **Staff Access:** View all bookings, update order status, manage time slots
- **Admin Access:** Full access including service CRUD and user management
- **API Restriction:** Spring Security @PreAuthorize annotations on all protected endpoints
- **UI Restriction:** Protected routes in React (PrivateRoute); role-aware screens in Android

Feature: Service Catalog

- **Screens:** Service Listing (Customer), Service Management (Admin)
- **Services:** Wash Only, Wash & Dry, Wash, Dry & Fold, Dry Clean, Iron Only
- **Fields:** Name, Description, Price per kg, Estimated Duration (hours), Active Status
- **API Endpoints:**
 - o GET /services,
 - o POST /services (Admin),
 - o PUT /services/{id} (Admin),
 - o DELETE /services/{id} (Admin)

Feature: Booking / Core Business Module

- **Screens:** Booking Form (Customer), Booking Dashboard (Staff/Admin)
- **Booking Fields:** Service, Time Slot, Estimated Weight (kg), Special Instructions
- **Status Workflow:**
 - o PENDING → RECEIVED → IN_PROGRESS → READY_FOR_PICKUP → COMPLETED / CANCELLED
- **Validation:** Slot availability check, capacity lock, no double-booking per user per slot
- **API Endpoints:**

- o POST /bookings,
 - o GET /bookings,
 - o GET /bookings/{id},
 - o PATCH /bookings/{id}/status,
 - o DELETE /bookings/{id}
- **Emails Sent:** Booking confirmation (on creation), Order ready notification (when status = READY_FOR_PICKUP)

Feature: Payment – PayMongo Integration

- **Mode:** Sandbox / Test Mode only
- **Provider:** PayMongo
- **Flow:**
 - o Create booking → redirect to PayMongo checkout → PayMongo webhook updates payment status → booking confirmed
- **Payment Methods:** GCash, Maya, Credit/Debit Card (as supported by PayMongo sandbox)
- **API Endpoints:**
 - o POST /payments/create,
 - o POST /payments/webhook
- **Database:** Payment record created with status PENDING; updated to PAID or FAILED via webhook

Feature: OpenWeatherMap Integration

- **Purpose:** Display current weather conditions on the customer dashboard as a contextual widget
- **API Used:** OpenWeatherMap Current Weather API
(api.openweathermap.org/data/2.5/weather)
- **Data Displayed:**
 - o City name,
 - o temperature (°C),
 - o weather condition (e.g., Cloudy),
 - o humidity,
 - o weather icon
- **Location:** Hardcoded to the shop city (e.g., Cebu City, PH)
- **Backend Proxy:** Spring Boot proxies the request to OpenWeatherMap to protect the API key
- **API Endpoint:** GET /weather

Feature: File Upload – Profile Picture

- **Purpose:** Allow customers to upload a profile picture linked to their account
- **Accepted Types:** JPEG, PNG
- **Max File Size:** 2 MB
- **Storage:** Stored on the Spring Boot server filesystem; file path saved in the users table
- **API Endpoint:** POST /users/{id}/avatar
- **Display:** Profile picture shown on the customer profile page and navbar avatar

Feature: Email Notifications (SMTP)

- **Provider:** SMTP via JavaMailSender (Gmail SMTP or Brevo as relay)
- **Email 1:** Welcome Email – sent on successful account registration
- **Email 2:** Booking Confirmation – sent immediately after a booking is successfully created and paid
- **Email 3:** Order Ready Notification – sent when staff updates order status to READY_FOR_PICKUP
- **Notes:** Console print is not accepted; real SMTP delivery required for all emails

Feature: Android Mobile Application

- **Technology:** Kotlin, XML-Based UI Layouts, Android API Level 34, Retrofit 2
- **Auth:** Login via email/password; JWT stored in EncryptedSharedPreferences
- **Screens:** Login, Register, Dashboard, Service List, Booking Form, Order Status, Profile
- **API Integration:** Retrofit 2 with OkHttp interceptor for Authorization header injection
- **Role Awareness:** Customer and Staff views rendered differently based on stored JWT role claim
- **Protected:** Unauthorized access to protected screens redirects to the login screen

2.5 Acceptance Criteria

AC-1: Successful User Registration

Scenario: A new customer creates an account using email/password.

- Given I am a new user on the registration page
- When I enter a valid name, email, address, contact number, and a password of at least 8 characters
- And click Register
- Then my account is created with a hashed password stored in the database
- And I receive a welcome email to my registered address

- And I am redirected to the login page

AC-2: Customer Successfully Books a Slot

Scenario: A logged-in customer completes the booking flow.

- Given I am logged in as a Customer
- When I select an available service and time slot and confirm the booking
- Then the slot capacity is decremented and locked to prevent double-booking
- And a booking record is created with status PENDING
- And I am redirected to the payment page
- And upon successful PayMongo payment, I receive a booking confirmation email
- And the booking appears in my order history

AC-3: Double-Booking Prevention

Scenario: A customer attempts to book a fully occupied time slot.

- Given a time slot has reached its maximum booking capacity
- When I attempt to book the same time slot
- Then the system returns HTTP 409 Conflict with error code SLOT-001
- And no booking record is created
- And the slot remains locked

AC-4: Staff Updates Order Status

Scenario: A staff member updates an order to Ready for Pickup.

- Given I am logged in as Staff
- When I update a booking status to READY_FOR_PICKUP
- Then the system sends an automated email notification to the customer
- And the order status is updated in the customer's dashboard
- And the change is recorded with a timestamp

AC-5: Admin Manages Services

Scenario: An admin adds a new laundry service.

- Given I am logged in as Admin
- When I submit a new service with name, description, price per kg, and duration
- Then the service is saved and appears in the customer service catalog
- And the service is available for selection during the booking flow

AC-6: Google OAuth Login

Scenario: A user logs in using Google.

- Given I am on the login page
- When I click 'Continue with Google' and authenticate with my Google account
- Then the system retrieves or creates my user record linked to the Google account
- And the system generates and returns a WashQ JWT
- And I am redirected to the customer dashboard

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: 95% of requests must complete within 2 seconds under normal load
- Web page initial load time: under 3 seconds on a standard broadband connection
- Android app cold start: under 3 seconds on a mid-range device
- Database queries: must complete within 500 milliseconds
- System must support at least 50 concurrent users during peak booking hours
- Slot locking during concurrent booking requests must be handled using database-level transactions to prevent race conditions

3.2 Security Requirements

- All API communication must use HTTPS in production
- JWT tokens must be signed with a secure secret key; tokens expire after 24 hours
- Passwords must be hashed using BCrypt with a minimum of 12 salt rounds
- API endpoints must enforce role-based authorization using Spring Security
- SQL injection must be prevented through JPA parameterized queries
- Cross-Site Scripting (XSS) must be mitigated through input sanitization and proper response headers
- Profile picture uploads must be validated for file type and size on the server side
- PayMongo webhook payloads must be verified using the webhook secret before processing
- Google OAuth tokens must be validated server-side before issuing a WashQ JWT

- JWT must be stored securely on Android using EncryptedSharedPreferences

3.3 Compatibility Requirements

- **Web Browsers:** Google Chrome, Mozilla Firefox, Microsoft Edge (latest 2 versions each)
- **Android:** API Level 34 (Android 14) as target; minimum API Level 26 (Android 8.0) for compatibility
- **Screen Sizes:**
 - Mobile (360px+),
 - Tablet (768px+),
 - Desktop (1024px+) — web application must be responsive
- **Backend:** Java 17+, Spring Boot 3.x
- **Database:** PostgreSQL 14+ hosted on Supabase

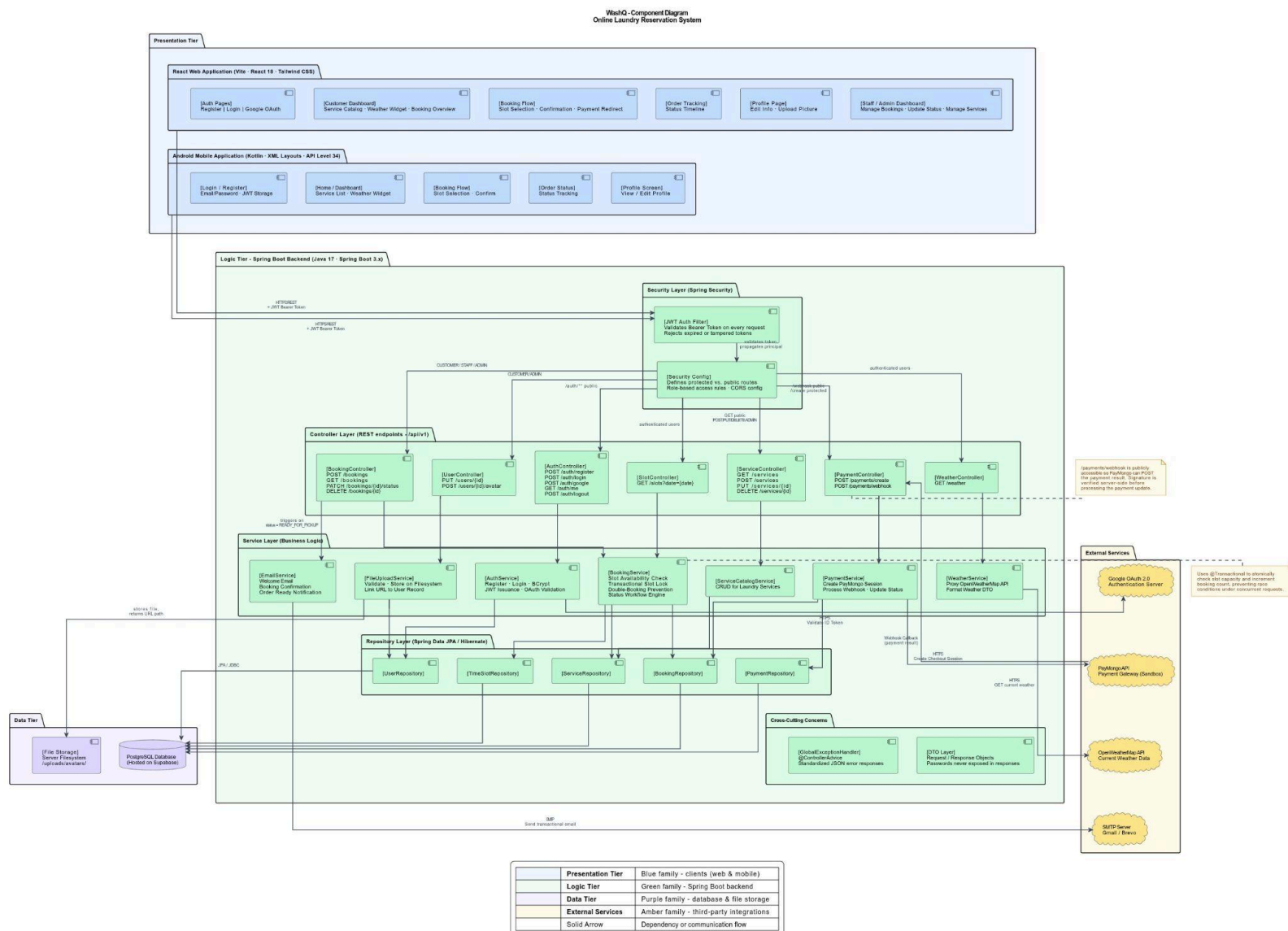
3.4 Usability Requirements

- A new customer must be able to complete their first booking within 5 minutes
- Error messages must be descriptive and actionable, guiding users toward correction
- Touch targets on the Android app must be a minimum of 48x48dp
- Navigation must be consistent and intuitive across all pages and screens
- The booking status workflow must be clearly communicated to customers through visual indicators (e.g., progress stepper)
- Forms must provide real-time inline validation feedback where possible

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

WashQ follows a Three-Tier Architecture (Presentation Tier, Logic Tier, Data Tier) with a Layered Backend Architecture pattern inside the Logic Tier. The system is composed of three presentation clients: the React Web Application, the Android Mobile Application, and the Admin/Staff Web Dashboard. All communicating exclusively with a single Spring Boot REST API backend. The backend persists all data to a PostgreSQL database hosted on Supabase and integrates with multiple third-party services.



Note: The image above has been resized for formatting purposes and may not be fully legible. For better clarity, please view the full-resolution image here:

ABEL_Component_Diagram.jpg

Architecture Pattern: Layered Architecture

The Spring Boot backend follows a strict Layered Architecture with the following layers:

- **Controller Layer** – Handles HTTP requests and responses; maps REST endpoints; validates incoming DTOs; delegates to the Service layer. No business logic resides here.
- **Service Layer** – Contains all business logic including slot availability checking, booking state transitions, payment processing coordination, email dispatch, and external API calls.
- **Repository Layer** – Interfaces with the PostgreSQL database via Spring Data JPA. Custom JPQL queries are defined here for complex lookups such as slot availability and booking history.
- **DTOs (Data Transfer Objects)** – Used to decouple the API contract from database entities. No entity with a password field is ever directly serialized in API responses.
- **Security Configuration** – Spring Security filter chain configuration; JWT filter; role-based access rules; CORS configuration.
- **Global Exception Handler** – @ControllerAdvice class that maps exceptions to standardized JSON error responses with proper HTTP status codes.

4.2 Technology Stack

Layer / Component	Technology
Backend	Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA, Hibernate
Database	PostgreSQL 14+ hosted on Supabase
Web Frontend	React 18, JavaScript, Tailwind CSS, Axios
Mobile	Kotlin, XML-Based UI Layouts, Retrofit 2, Android API Level 34
Authentication	JWT (Spring Security), BCrypt, Google OAuth 2.0
Payment	PayMongo API (Sandbox Mode)
Email	SMTP via JavaMailSender (e.g., Gmail SMTP / Brevo)
External API	OpenWeatherMap REST API
File Storage	Local server storage (Spring Boot MultipartFile)
Build Tools	Maven (Backend), npm (Web), Gradle (Android)
Version Control	Git / GitHub

4.3 Backend Module Structure

The Spring Boot project is organized into the following package structure:

```
com.washq.backend
├── controller/      # REST controllers (AuthController, BookingController, etc.)
├── service/         # Business logic (BookingService, PaymentService, EmailService, etc.)
├── repository/      # JPA repositories (UserRepository, BookingRepository, etc.)
├── entity/          # JPA entities (User, Booking, Service, TimeSlot, Payment)
├── dto/             # Request/Response DTOs
├── security/        # JWT filter, UserDetailsService, SecurityConfig
├── exception/       # Custom exceptions & GlobalExceptionHandler
├── integration/     # External API clients (WeatherClient, PayMongoClient)
└── config/          # App-level configs (CORS, Mail, File Upload)
```

Each layer communicates strictly downward: Controller → Service → Repository. Cross-layer skipping is not permitted. External integrations (PayMongo, OpenWeatherMap, SMTP) are encapsulated in dedicated service classes under the integration package.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Field	Value
Base URL	http://localhost:8080/api/v1
Format	JSON for all requests and responses
Authentication	Bearer token (JWT) in Authorization header

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Register a new customer account with email and password
API URL	POST /auth/register
Authentication	None
Request Payload	<pre>{ "firstName": "<string>", "lastName": "<string>", "email": "<email>", "password": "<string>", "address": "<string>", "contactNumber": "<string>" }</pre>
Success Response (201)	<pre>{ "success": true, "data": { "user": { "id", "email", "firstName", "lastName", "role" }, "accessToken": "<jwt>" }, "error": null, "timestamp": "..." }</pre>
Notes	Triggers welcome email to the registered address. Password is hashed before storage.
Description	Register a new customer account with email and password

User Login

Description	Authenticate with email and password and receive a JWT
API URL	POST /auth/login
Authentication	None
Request Payload	{ "email": "<email>", "password": "<string>" }
Success Response (200)	{ "success": true, "data": { "user": { "id", "email", "firstName", "lastName", "role" }, "accessToken": "<jwt>" }, "error": null, "timestamp": "..." }
Error	AUTH-001 (401) if credentials are invalid
Description	Authenticate with email and password and receive a JWT

Get Current Authenticated User

Description	Retrieve the authenticated user's profile from the JWT
API URL	GET /auth/me
Authentication	Bearer Token (JWT) required
Request Payload	None
Success Response (200)	{ "success": true, "data": { "id", "email", "firstName", "lastName", "address", "contactNumber", "role", "profilePictureUrl" }, "error": null, "timestamp": "..." }
Notes	Password is never included in the response (DTO pattern).
Description	Retrieve the authenticated user's profile from the JWT

Google OAuth Login

Description	Exchange a Google ID token for a WashQ JWT; creates user if not found
API URL	POST /auth/google
Authentication	None (Google ID token sent in body)
Request Payload	{ "idToken": "<google_id_token>" }

Success Response (200/201)	Same as User Login response with WashQ-issued JWT
Notes	If this is the user's first OAuth login, a new account is created. Returns 201 Created for new users, 200 OK for returning users.

Logout

Description	Invalidate the current session (client-side token deletion)
API URL	POST /auth/logout
Authentication	Bearer Token (JWT) required
Request Payload	None
Success Response (200)	{ "success": true, "data": { "message": "Logged out successfully" } }
Notes	JWT is stateless; logout is primarily handled client-side by deleting the stored token.

User Endpoints

Update User Profile

Description	Update the authenticated user's profile information
API URL	PUT /users/{id}
Authentication	Bearer Token (JWT) required; users can only update their own profile
Request Payload	{ "firstName": "<string>", "lastName": "<string>", "address": "<string>", "contactNumber": "<string>" }
Success Response (200)	Returns updated user DTO
Error	AUTH-003 (403) if attempting to update another user's profile without admin role

Upload Profile Picture

Description	Upload a profile picture for the authenticated user
API URL	POST /users/{id}/avatar
Authentication	Bearer Token (JWT) required
Request Payload	multipart/form-data with field name 'file' (JPEG or PNG, max 2 MB)
Success Response (200)	{ "success": true, "data": { "profilePictureUrl": "<url>" } }
Error	FILE-001 (400) if file type is not JPEG/PNG or exceeds 2 MB

Service Endpoints

Get All Services

Description	Retrieve the list of all active laundry services
API URL	GET /services
Authentication	None (public endpoint)
Success Response (200)	Array of service objects: { id, name, description, pricePerKg, estimatedDurationHours, isActive }

Create Service (Admin Only)

Description	Add a new laundry service to the catalog
API URL	POST /services
Authentication	Bearer Token; ADMIN role required
Request Payload	{ "name": "<string>", "description": "<string>", "pricePerKg": <decimal>, "estimatedDurationHours": <int> }
Success Response (201)	Created service object
Error	AUTH-003 (403) if role is not ADMIN; VALID-001 (400) if fields are missing

Update Service (Admin Only)

Description	Update an existing laundry service
API URL	PUT /services/{id}
Authentication	Bearer Token; ADMIN role required
Request Payload	Same as Create Service
Success Response (200)	Updated service object

Delete Service (Admin Only)

Description	Soft-delete (deactivate) a laundry service
API URL	DELETE /services/{id}
Authentication	Bearer Token; ADMIN role required
Success Response (200)	{ "success": true, "data": { "message": "Service deactivated." } }

Time Slot Endpoints

Get Available Time Slots

Description	Retrieve available time slots for a specific date
API URL	GET /slots?date={YYYY-MM-DD}
Authentication	Bearer Token required
Success Response (200)	Array of slot objects: { id, date, startTime, endTime, maxCapacity, currentBookingCount, isAvailable }
Notes	Only slots with currentBookingCount < maxCapacity and isAvailable = true are returned as bookable.

Booking Endpoints

Create Booking

Description	Create a new laundry booking. Locks the slot and initiates a payment.
API URL	POST /bookings
Authentication	Bearer Token; CUSTOMER role required
Request Payload	{ "servicId": <int>, "slotId": <int>, "estimatedWeightKg": <decimal>, "specialInstructions": "<string>" }
Success Response (201)	Created booking object with status PENDING and a PayMongo checkout URL
Error	SLOT-001 (409) if slot is full; SLOT-002 (409) if user already has a booking in this slot

Get My Bookings (Customer) / All Bookings (Staff-Admin)

Description	Returns bookings filtered by authenticated user's role
API URL	GET /bookings
Authentication	Bearer Token required
Behavior	CUSTOMER: returns only their own bookings. STAFF/ADMIN: returns all bookings.
Success Response (200)	Array of booking objects with full details

Update Booking Status (Staff/Admin Only)

Description	Advance a booking through the status workflow
API URL	PATCH /bookings/{id}/status
Authentication	Bearer Token; STAFF or ADMIN role required
Request Payload	{ "status": "RECEIVED IN_PROGRESS READY_FOR_PICKUP COMPLETED CANCELLED" }
Success Response (200)	Updated booking object
Notes	Setting status to READY_FOR_PICKUP automatically triggers an order-ready email to the customer.

Cancel Booking (Customer)

Description	Customer cancels their own booking
API URL	DELETE /bookings/{id}
Authentication	Bearer Token; booking must belong to the authenticated user
Success Response (200)	Booking status updated to CANCELLED; slot capacity freed
Error	AUTH-003 (403) if booking belongs to another user; DB-001 (404) if booking not found

Payment Endpoints

Create Payment

Description	Create a PayMongo checkout session for a booking
API URL	POST /payments/create
Authentication	Bearer Token required
Request Payload	{ "bookingId": <int> }
Success Response (201)	{ "success": true, "data": { "paymentId": <int>, "checkoutUrl": "<paymongo_url>" } }
Notes	The checkout URL is returned to the client for redirect to PayMongo's payment page.

PayMongo Webhook

Description	Receives payment status updates from PayMongo after checkout completion
API URL	POST /payments/webhook
Authentication	PayMongo webhook signature header verified server-side
Behavior	On payment success: updates payment status to PAID and booking status to RECEIVED. On failure: updates payment status to FAILED.
Notes	This endpoint must be publicly accessible and registered in the PayMongo dashboard.

Weather Endpoint

Get Current Weather

Description	Returns current weather data for the shop's city, proxied from OpenWeatherMap
API URL	GET /weather
Authentication	Bearer Token required
Success Response (200)	{ "success": true, "data": { "city": "Cebu City", "temperature": 31.5, "condition": "Partly Cloudy", "humidity": 78, "iconUrl": "<openweathermap_icon_url>" }, "error": null }
Notes	The OpenWeatherMap API key is stored as a backend environment variable and is never exposed to the client.

5.3 Error Handling

HTTP Status Codes

Status Code	Meaning
200 OK	Successful request
201 Created	Resource successfully created
400 Bad Request	Invalid input or failed validation
401 Unauthorized	Authentication required or token invalid
403 Forbidden	Authenticated but insufficient role permissions
404 Not Found	Requested resource does not exist
409 Conflict	Duplicate resource or slot conflict (double-booking)
500 Internal Server Error	Unexpected server error

Error Response Examples

Example 1 – Invalid Credentials:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "AUTH-001",
    "message": "Invalid credentials",
    "details": "Email or password is incorrect"
  },
  "timestamp": "2026-02-28T10:30:00Z"
}
```

Example 2 – Slot Unavailable:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "SLOT-001",
    "message": "Slot unavailable",
    "details": "The selected time slot has reached its maximum booking capacity"
  },
  "timestamp": "2026-02-28T10:35:00Z"
}
```

Example 3 – Validation Failed:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Validation failed",
    "details": {
      "email": "Email is required",
      "password": "Password must be at least 8 characters"
    }
  },
  "timestamp": "2026-02-28T10:40:00Z"
}
```

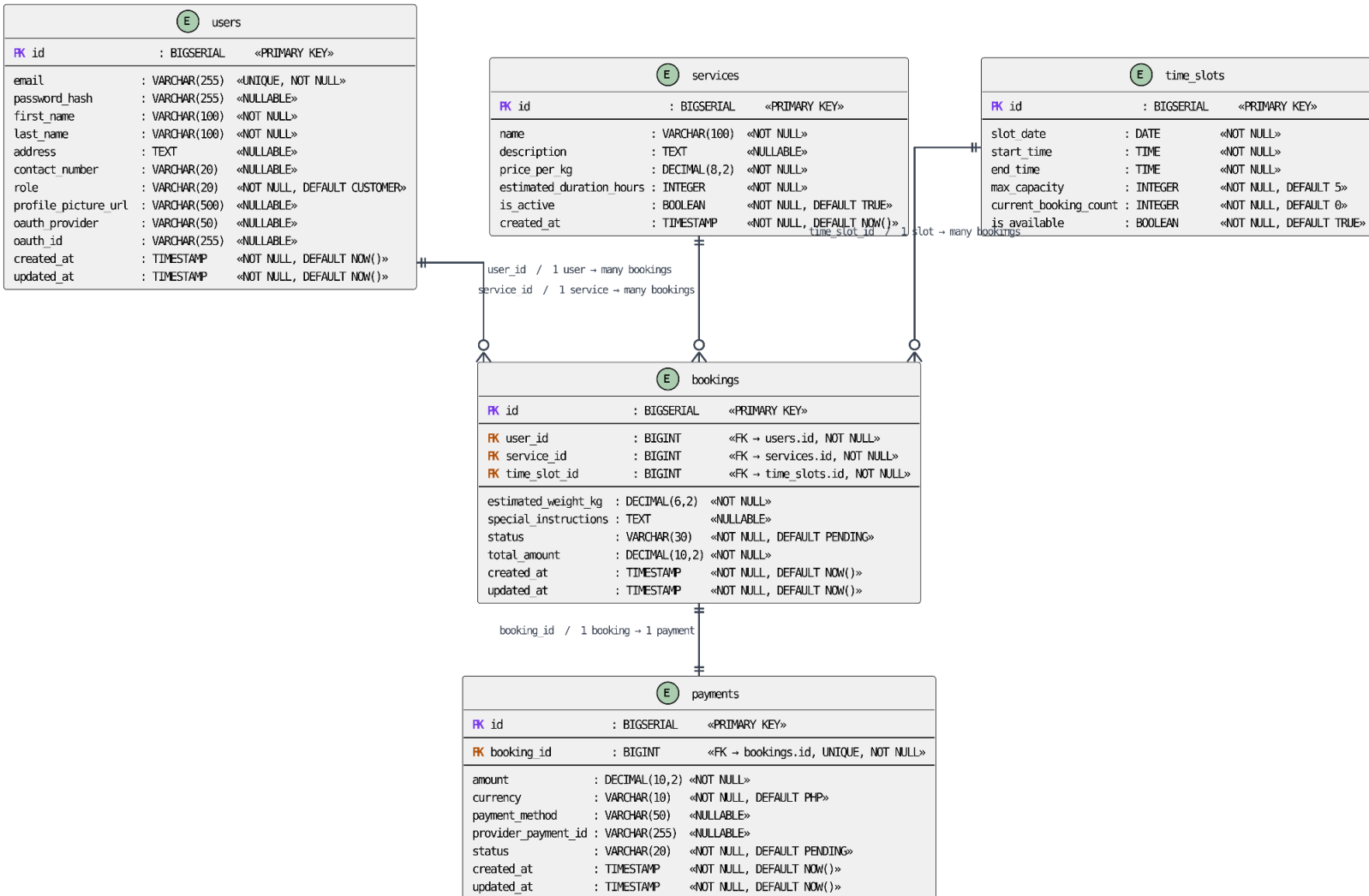
Error Code Reference

Error Code	Message	Details
AUTH-001	Invalid credentials	Email or password is incorrect
AUTH-002	Token expired	JWT token has expired, re-login required
AUTH-003	Insufficient permissions	User role does not allow this action
AUTH-004	OAuth login failed	Google OAuth authentication failed
VALID-001	Validation failed	One or more input fields are invalid
SLOT-001	Slot unavailable	Selected time slot is full or already taken
SLOT-002	Double-booking prevented	User already has a booking in this time slot
DB-001	Resource not found	Requested entity does not exist in the database
DB-002	Duplicate entry	A record with this data already exists
PAY-001	Payment failed	PayMongo payment could not be processed
PAY-002	Payment already recorded	Booking has already been paid
FILE-001	Invalid file type	Only JPEG and PNG image files are accepted
SYSTEM-001	Internal server error	Unexpected server-side error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

WashQ - Entity Relationship Diagram
OnLine Laundry Reservation System



Note: The image above has been resized for formatting purposes and may not be fully legible. For better clarity, please view the full-resolution image here:

ABEL_ERD.jpg

6.2 Entity Relationships

- **One-to-Many:** User → Bookings (one user can have many bookings over time)
- **Many-to-One:** Booking → Service (many bookings may reference the same laundry service)
- **Many-to-One:** Booking → TimeSlot (many bookings can share a time slot up to its max capacity)

- **One-to-One:** Booking → Payment (each booking has exactly one associated payment record)
- **Implicit:** User stores the `profile_picture_url` as a string path to the uploaded file on the server

6.3 Database Tables

Table: users

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing user ID
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address (login identifier)
password_hash	VARCHAR(255)	NULLABLE	BCrypt hashed password; NULL for OAuth-only users
first_name	VARCHAR(100)	NOT NULL	User's first name
last_name	VARCHAR(100)	NOT NULL	User's last name
address	TEXT	NULLABLE	User's home or delivery address
contact_number	VARCHAR(20)	NULLABLE	Philippine mobile or landline number
role	VARCHAR(20)	NOT NULL, DEFAULT CUSTOMER	CUSTOMER, STAFF, or ADMIN
profile_picture_url	VARCHAR(500)	NULLABLE	Server path to uploaded profile picture
oauth_provider	VARCHAR(50)	NULLABLE	e.g., 'google' if registered via OAuth
oauth_id	VARCHAR(255)	NULLABLE	Google subject ID for OAuth-linked accounts
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Account creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last profile update timestamp

Table: services

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing service ID
name	VARCHAR(100)	NOT NULL	Service name (e.g., Wash & Dry)
description	TEXT	NULLABLE	Description shown to customers
price_per_kg	DECIMAL(8,2)	NOT NULL	Price in PHP per kilogram
estimated_duration_hours	INTEGER	NOT NULL	Estimated turnaround time in hours
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete flag; false hides from catalog
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp

Table: time_slots

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing slot ID
slot_date	DATE	NOT NULL	Calendar date of the slot
start_time	TIME	NOT NULL	Slot start time (e.g., 08:00)
end_time	TIME	NOT NULL	Slot end time (e.g., 10:00)
max_capacity	INTEGER	NOT NULL, DEFAULT 5	Maximum bookings allowed in this slot
current_booking_count	INTEGER	NOT NULL, DEFAULT 0	Number of active bookings in this slot
is_available	BOOLEAN	NOT NULL, DEFAULT TRUE	Manual override to disable a slot

Table: bookings

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing booking ID
user_id	BIGINT	FK → users.id, NOT NULL	Customer who made the booking
service_id	BIGINT	FK → services.id, NOT NULL	Selected laundry service
time_slot_id	BIGINT	FK → time_slots.id, NOT NULL	Selected time slot
estimated_weight_kg	DECIMAL(6,2)	NOT NULL	Estimated laundry weight in kg
special_instructions	TEXT	NULLABLE	Optional notes from the customer
status	VARCHAR(30)	NOT NULL, DEFAULT PENDING	PENDING, RECEIVED, IN_PROGRESS, READY_FOR_PICKUP, COMPLETED, CANCELLED
total_amount	DECIMAL(10,2)	NOT NULL	Computed: price_per_kg × estimated_weight_kg
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Booking creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last status update timestamp

Table: payments

Column	Type	Constraints	Description
id	BIGSERIAL	PRIMARY KEY	Auto-incrementing payment ID
booking_id	BIGINT	FK → bookings.id, UNIQUE, NOT NULL	The booking this payment belongs to
amount	DECIMAL(10,2)	NOT NULL	Payment amount in PHP
currency	VARCHAR(10)	NOT NULL, DEFAULT PHP	Currency code
payment_method	VARCHAR(50)	NULLABLE	e.g., gcash, card (set on webhook)
provider_payment_id	VARCHAR(255)	NULLABLE	PayMongo payment or checkout session ID
status	VARCHAR(20)	NOT NULL, DEFAULT PENDING	PENDING, PAID, FAILED, REFUNDED
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Payment record creation timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last payment status update timestamp

6.4 Database Design Notes

- All passwords are stored as BCrypt hashes. Plain-text passwords are never persisted.
- DTOs are used for all API responses; entity classes with password_hash are never directly serialized.
- OAuth users have a NULL password_hash; they authenticate exclusively through the Google OAuth flow.
- Slot locking is implemented using a database transaction that increments current_booking_count and checks against max_capacity atomically, preventing race conditions under concurrent requests.
- Soft deletion is used for services (is_active = false) to preserve historical booking records without breaking foreign key constraints.

- The total_amount in bookings is computed at booking creation time to preserve the price at the time of booking, independent of future price changes to the service.

7.0 UI/UX DESIGN

7.1 Web Application Design System

7.1.1 Brand Identity

WashQ is positioned as clean, modern, and trustworthy which is approachable for university students and everyday residents. The visual language is fresh and airy, avoiding both corporate stiffness and childish aesthetics. The name evokes water and queuing; the design reflects efficiency and clarity.

7.1.2 Color Palette

Primary #1D4ED8	Primary Light #DBEAFE	Buttons, CTAs, active states / Backgrounds, cards, highlights
Primary Dark #1E3A8A	Accent #0EA5E9	Headers, nav, hover states / Badges, status indicators, icons
Success #16A34A	Warning #D97706	Order ready, payment confirmed / Pending states, slot filling up
Error #DC2626	Neutral 50 #F8FAFC	Failed payment, validation errors / Page backgrounds
Neutral 100 #F1F5F9	Neutral 400 #94A3B8	Card backgrounds / Placeholder text, disabled states
Neutral 700 #334155	Neutral 900 #0F172A	Body text / Headings

7.1.3 Typography

Font Family	Inter (Google Fonts) — loaded via CDN in index.html
Heading 1	32px / Bold / #0F172A — Page titles
Heading 2	24px / SemiBold / #0F172A — Section titles
Heading 3	18px / SemiBold / #334155 — Card headers, subsections
Body	14px / Regular / #334155 — Default paragraph and label text
Small / Caption	12px / Regular / #94A3B8 — Timestamps, helper text, secondary labels

Button	14px / SemiBold / Uppercase letter-spacing — All button labels
---------------	--

7.1.4 Spacing System

Base unit: 4px. All spacing values are multiples: 4px, 8px, 12px, 16px, 24px, 32px, 48px, 64px. Component internal padding follows the card token (24px). Section vertical spacing is 32–48px.

7.1.5 Component Tokens

Primary Button	bg #1D4ED8 · text white · radius 8px · padding 10px 20px · hover bg #1E3A8A · active scale 0.97
Secondary Button	border 1.5px #1D4ED8 · text #1D4ED8 · bg transparent · hover bg #DBEAFE
Danger Button	bg #DC2626 · text white · radius 8px · used for destructive actions only
Input Field	border #CBD5E1 · radius 8px · focus border #1D4ED8 · focus ring 3px #DBEAFE · error border #DC2626
Card	bg white · radius 16px · shadow 0 1px 3px rgba(0,0,0,0.1) · padding 24px
Elevated Card	shadow 0 4px 12px rgba(0,0,0,0.1) · used for modals, dropdowns
Badge — Pending	bg #FEF3C7 · text #D97706 · radius 9999px · 12px SemiBold · padding 4px 10px
Badge — In Progress	bg #DBEAFE · text #1D4ED8 · radius 9999px · 12px SemiBold
Badge — Ready for Pickup	bg #DCFCE7 · text #16A34A · radius 9999px · 12px SemiBold
Badge — Completed	bg #F1F5F9 · text #334155 · radius 9999px · 12px SemiBold
Badge — Cancelled	bg #FEE2E2 · text #DC2626 · radius 9999px · 12px SemiBold

7.1.6 Layout System

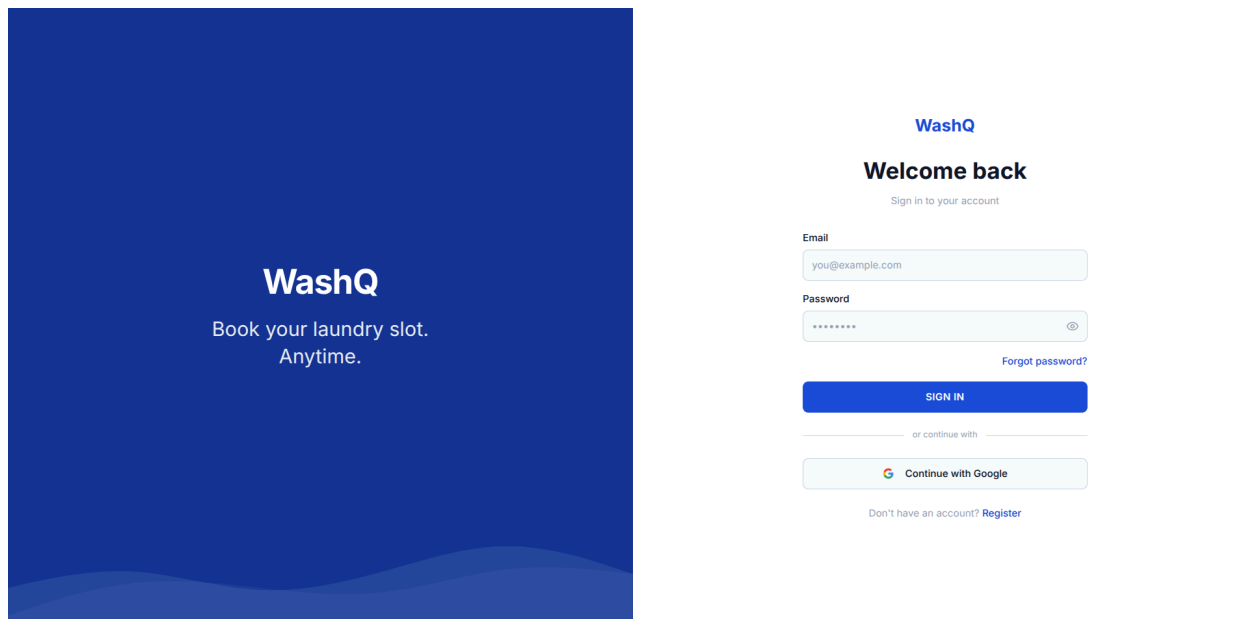
Max Content Width	1280px, centered with auto horizontal margins
Top Navbar (Customer)	64px height · bg #1E3A8A · full viewport width · sticky on scroll

Sidebar (Staff/Admin)	240px fixed left · bg #0F172A · full viewport height · collapses to icon rail on tablet
Responsive Breakpoints	Mobile: 640px · Tablet: 768px · Desktop: 1024px · Large: 1280px
Grid System	12-column grid on desktop · 8-column on tablet · 4-column on mobile
Z-index Scale	Navbar: 50 · Sidebar: 40 · Modal overlay: 60 · Toast/Notifications: 70

7.2 Web Application Screens

The following ten screens cover the complete customer-facing and staff/admin-facing web application. Each screen description includes layout structure, component inventory, interaction states, and micro-interaction notes for Figma implementation.

Screen W-01 Login Page [Public — All Users]



Overview

The Login page is the entry point for all authenticated flows. It is a centered, split-layout page: a decorative brand panel on the left and the login form on the right. The page is publicly accessible and redirects authenticated users to their role-appropriate dashboard.

Layout Structure

Page Background	Full viewport · bg #F8FAFC
Left Panel	50% viewport width · bg #1E3A8A · hidden below tablet breakpoint (768px)
Left Panel Content	Vertically and horizontally centered · WashQ logo (white) · tagline 'Book your laundry slot. Anytime.' in Inter 28px white · decorative wave/water illustration at bottom
Right Panel	50% viewport width (100% on mobile) · bg white · flex column centered · padding 48px 64px on desktop, 32px 24px on mobile
Form Container	Max-width 400px · centered within right panel

Components

- WashQ wordmark — Inter Bold 24px #1D4ED8, positioned at top of form container
- Page heading — 'Welcome back' · H1 · #0F172A
- Subheading — 'Sign in to your account' · Body 14px · #94A3B8
- Email input field — label 'Email address' · type email · placeholder 'you@example.com'
- Password input field — label 'Password' · type password · toggle show/hide icon (eye icon, right side)
- Forgot password? — right-aligned text link · 12px · #1D4ED8 · below password field
- Primary button — full-width · 'SIGN IN' · bg #1D4ED8
- Divider row — 'or continue with' text centered between two horizontal rules · #94A3B8
- Google OAuth button — full-width secondary button · Google 'G' logo SVG icon left · 'Sign in with Google'
- Registration link — 'Don't have an account? Register' · centered below Google button · 'Register' is a #1D4ED8 link

States

- Default — form empty, all fields in neutral state
- Loading — on form submit: button shows spinner (circular, white, 16px) replacing the label text, button disabled, inputs disabled
- Error — inline error banner below form: bg #FEE2E2 · border-left 4px #DC2626 · text 'Invalid email or password.' · 14px · #DC2626 · icon: x-circle
- Validation — real-time inline errors below each field on blur: red 12px caption text + input border turns #DC2626

- Success — brief green checkmark flash before redirect; no separate success screen

Micro-interactions

- Password toggle: eye icon animates between open/closed state on click; smooth 150ms opacity transition
- Sign In button: scale(0.97) press effect on active; ripple-style hover darkens to #1E3A8A over 200ms
- Input focus: border transitions from #CBD5E1 to #1D4ED8 over 150ms with a soft #DBEAFE focus ring expanding outward
- Error banner slides down from top of form container with a 200ms ease-out animation
- Redirect after login: fade-out transition on the right panel before navigating

Screen W-02 Registration Page [Public — New Users]

WashQ
Book your laundry slot.
Anytime.

WashQ
Create your account
Join WashQ and start booking today.

First Name: Juan
Last Name: Dela Cruz
Email: you@example.com
Password: *****
Address (Optional): Your address
Contact Number: +63 9XX XXX XXXX

CREATE ACCOUNT

or

Sign up with Google

Already have an account? [Sign in](#)

Overview

The Registration page uses the same split-layout as the Login page for visual continuity. The form is longer (six fields) and uses a two-column grid for name fields on desktop. On successful registration, a welcome email is dispatched and the user is redirected to the Customer Dashboard.

Layout Structure

Page Background	Full viewport · bg #F8FAFC
Left Panel	50% viewport width · bg #1E3A8A · same brand panel as Login · hidden below 768px
Right Panel	50% viewport width (100% on mobile) · bg white · vertically scrollable · padding 48px 64px desktop, 32px 24px mobile
Form Container	Max-width 480px · centered
Name Row	Two equal columns (gap 16px) on desktop · stacked on mobile

Components

- WashQ wordmark — same as Login
- Page heading — 'Create your account' · H1
- Subheading — 'Join WashQ and start booking today.' · Body · #94A3B8
- First Name input — left column of name row · required
- Last Name input — right column of name row · required
- Email input — full width · required
- Password input — full width · with show/hide toggle · required · helper text below: '8 characters minimum'
- Address textarea — full width · 3 rows · placeholder 'Your home or pickup address' · optional
- Contact Number input — full width · placeholder '+63 9XX XXX XXXX' · optional
- Primary button — full-width · 'CREATE ACCOUNT'
- Divider — 'or' separator
- Google OAuth button — full-width · 'Sign up with Google'
- Login link — 'Already have an account? Sign in' · #1D4ED8 link

States

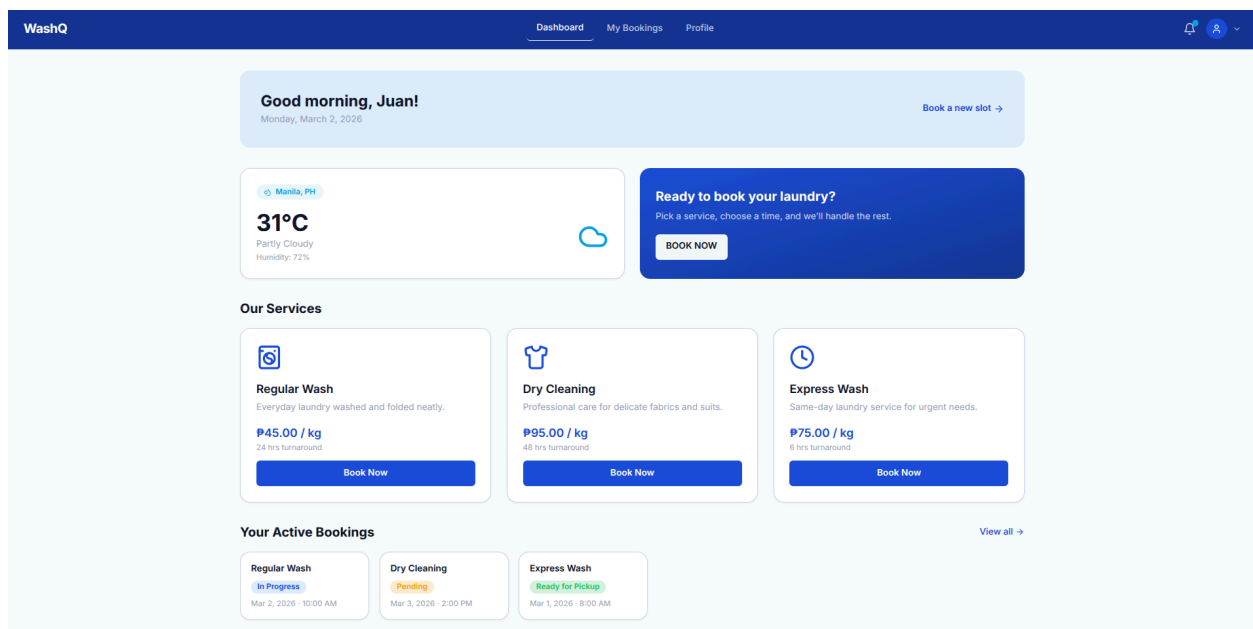
- Default — all fields empty
- Inline validation — each required field validates on blur; shows red caption and error border
- Password strength indicator — horizontal bar below password field: red (weak, < 8 chars), yellow (moderate), green (strong, 8+ with mixed chars); transitions smoothly on each keystroke
- Loading — 'CREATE ACCOUNT' button shows spinner; all fields and buttons disabled
- Error — error banner: 'This email is already registered. Sign in instead?' with 'Sign in' as inline link

- Success — full-width success toast appears at top of page: bg #DCFCE7 · 'Account created! Check your email for a welcome message.' · auto-dismisses after 4 seconds · user is then navigated to dashboard

Micro-interactions

- Password strength bar: width and color animate on each keypress (transition: width 200ms, color 200ms)
- Two-column name fields collapse to single column below 640px with a smooth height transition
- Google button hover: background transitions to #DBEAFE over 200ms, subtle shadow lifts
- Success toast slides in from top with ease-out 300ms, fades out after 3.5 seconds

Screen W-03 Customer Dashboard [Authenticated — Customer Role]



Overview

The Customer Dashboard is the main hub after login. It presents three primary content zones: a top hero/welcome bar, a weather widget alongside a quick booking CTA, and the service catalog. A current bookings overview strip is anchored at the bottom. The layout uses the top navbar and a single-column content area on mobile, expanding to a multi-column grid on desktop.

Layout Structure

Page Background	bg #F8FAFC
Top Navbar	64px · bg #1E3A8A · sticky · contains: WashQ logo (left), nav links (Dashboard, My Bookings, Profile), notification bell icon, avatar dropdown (right)
Content Area	Max-width 1280px · centered · padding 32px 24px
Welcome Banner	Full-width · bg #DBEAFE · radius 16px · padding 32px · greeting + date
Info Row	Two-column grid: Weather Widget (left, 40%) + Quick Book CTA card (right, 60%) · gap 24px · stacks on mobile
Services Section	Full-width · section heading + 3-column card grid on desktop, 2-col tablet, 1-col mobile
Current Bookings Strip	Full-width · section heading + horizontal scroll row of booking summary cards

Components

Top Navbar

- WashQ logo — white wordmark, left
- Navigation links — 'Dashboard' (active state: underline + white), 'My Bookings', 'Profile' · Inter 14px SemiBold · text white/70% inactive
- Notification bell icon — 24px · white · badge dot (#0EA5E9) when unread notifications exist
- Avatar dropdown — circular avatar 36px (profile picture or initials fallback on #1D4ED8 bg) · on click: dropdown card with 'My Profile', 'My Bookings', divider, 'Sign Out'

Welcome Banner

- Greeting — 'Good morning, [First Name]!' · H2 · #0F172A
- Date line — 'Saturday, February 28, 2026' · Body 14px · #94A3B8
- CTA link — 'Book a new slot →' · #1D4ED8 · 14px SemiBold · right-aligned within banner

Weather Widget

- Card (bg white, radius 16px, shadow card)
- Location chip — sky blue #0EA5E9 badge pill · 'Cebu City, PH'
- Temperature — 36px Bold #0F172A · e.g., '31°C'
- Condition — 'Partly Cloudy' · H3 · #334155
- Weather icon — OpenWeatherMap icon or custom SVG, 48px

- Humidity row — small cloud-rain icon + '78% Humidity' · Caption 12px · #94A3B8
- Subtle laundry advisory — 'Good day to do laundry!' or 'Consider indoor drying today.' based on condition · italic 12px · #94A3B8

Quick Book CTA Card

- Card (bg #1D4ED8 gradient to #1E3A8A, radius 16px, elevated shadow)
- Headline — 'Ready to book your laundry?' · H2 · white
- Subtext — 'Choose a service and reserve your slot in under 2 minutes.' · Body · white/80%
- Button — 'BOOK NOW' · secondary style inverted (white bg, #1D4ED8 text) · radius 8px · right-aligned

Services Section

- Section heading — 'Our Services' · H2
- Service Card (per service): white card · radius 16px · shadow · padding 24px
 - Service icon — custom SVG, 40px, color #1D4ED8
 - Service name — H3 · #0F172A · e.g., 'Wash & Dry'
 - Description — Body 14px · #94A3B8 · max 2 lines, truncated
 - Price — '₹ [X].00 / kg' · 18px SemiBold · #1D4ED8
 - Duration — '~X hours' · Caption · #94A3B8
 - 'Book Now' button — Primary · full-width inside card

Current Bookings Strip

- Section heading — 'Your Active Bookings' + 'View all →' link · right-aligned
- Horizontal scroll container of booking summary cards (200px wide each)
- Booking card: service name · status badge · slot date/time · 'View Details' link
- Empty state: centered illustration + 'No active bookings yet.' + 'Book Now' button

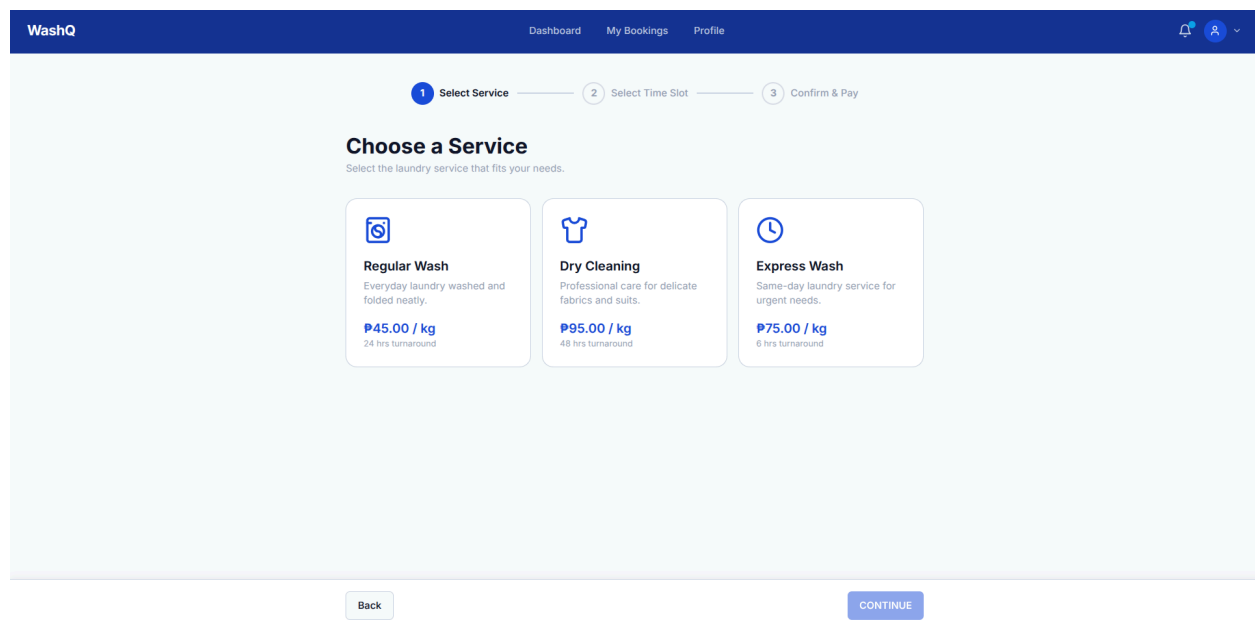
States

- Loading — skeleton placeholders: shimmer grey bars in place of weather data, service cards, and booking strip; navbar loads fully first
- Weather API error — weather card shows: cloud-off icon + 'Weather unavailable' + 'Check back later' caption; no broken layout
- No services — services section shows empty state card: 'No services available at the moment.'
- No bookings — booking strip empty state as described above

Micro-interactions

- Service cards: lift on hover — shadow transitions from card shadow to elevated shadow (200ms), translateY(-2px)
- Avatar dropdown: scales in from top-right origin on open (200ms ease-out), fades out on close
- Notification badge: pulses once on mount if there are unread notifications (scale 1.0→1.3→1.0 over 600ms)
- Weather widget: temperature number counts up from 0 on first load (300ms ease-out) — communicates freshness of data
- Navbar: on scroll past 64px, adds a subtle drop shadow (0 2px 8px rgba(0,0,0,0.2)) to reinforce sticky layering

Screen W-04 Booking Flow — Step 1: Select Service [Authenticated — Customer Role]



Overview

The booking flow is a three-step wizard. A persistent progress stepper at the top tracks position. Step 1 presents all active laundry services as selectable cards. Selecting a card highlights it and enables the 'Continue' button.

Layout Structure

Page Background	bg #F8FAFC
Navbar	Same sticky top navbar as Dashboard

Content Area	Max-width 860px · centered · padding 40px 24px
Progress Stepper	Full-width · 3 steps · Step 1 active
Service Grid	3-column on desktop · 2-column on tablet · 1-column on mobile · gap 16px
Footer Bar	Sticky bottom bar · bg white · border-top #F1F5F9 · shadow elevated · contains Back + Continue buttons

Components

Progress Stepper

- Three numbered circles connected by lines: '1 Select Service' (active: #1D4ED8 filled circle, white number) · '2 Select Time Slot' (inactive: #94A3B8 circle) · '3 Confirm & Pay' (inactive)
- Step labels: Caption 12px below each circle
- Connector lines: #CBD5E1 inactive, #1D4ED8 completed

Step Heading

- 'Choose a Service' · H1 · #0F172A
- Subtext — 'Select the laundry service that fits your needs.' · Body · #94A3B8

Service Selection Cards

- Same card layout as Dashboard service cards with an additional selection state
- Unselected: white card · #CBD5E1 border 1.5px
- Selected: card border becomes #1D4ED8 2.5px · bg #F0F7FF · checkmark icon (top-right corner, #1D4ED8, 20px)
- Only one card can be selected at a time (radio-group behavior)

Sticky Footer Bar

- 'Back' — secondary button (left) · navigates to previous page
- 'CONTINUE' — primary button (right) · disabled (opacity 40%, not clickable) until a service is selected · enabled on selection

States

- Default — no service selected; 'CONTINUE' button disabled
- Selected — one card highlighted; 'CONTINUE' enabled
- Loading services — skeleton cards (shimmer); footer bar hidden until loaded

- Empty services — full-width message: 'No services are currently available. Please check back later.'

Micro-interactions

- Card selection: border color and background transition over 150ms; checkmark icon fades in from opacity 0 + scale(0.8) to opacity 1 + scale(1)
- 'CONTINUE' button: transitions from disabled to enabled state with opacity change over 150ms
- Stepper connector line fills with #1D4ED8 left-to-right on step completion using a CSS width transition

Screen W-05 Booking Flow — Step 2: Select Time Slot [Authenticated — Customer Role]

The screenshot displays the 'Select Time Slot' step of the WashQ booking process. At the top, a progress bar shows three steps: 'Select Service' (completed), 'Select Time Slot' (current), and 'Confirm & Pay'. Below this, a date picker shows 'Mon 2' as the selected date. A grid of time slots for Monday, March 2, is presented. The slots are: 8:00 AM - 10:00 AM (3 of 5 spots left), 10:00 AM - 12:00 PM (Filling up, 1 of 5 left), 12:00 PM - 2:00 PM (Fully Booked), 2:00 PM - 4:00 PM (4 of 5 spots left), 4:00 PM - 6:00 PM (2 of 5 spots left), and 6:00 PM - 8:00 PM (Fully Booked). A 'Booking Summary' sidebar on the right shows 'Regular Wash' at P45.00 / kg, an 'Estimated Weight' of 3 kg, and an 'Estimated Total' of P135.00. At the bottom, there are 'Back' and 'CONTINUE' buttons.

Overview

Step 2 presents a date picker and a grid of available time slots for the selected date. Slots at or near capacity show a warning indicator. Fully booked slots are visually disabled. A summary sidebar on the right shows the selected service details on desktop.

Layout Structure

Content Area	Max-width 1024px · two-column on desktop (slot grid 65% + summary sidebar 35%) · stacks to single column on tablet/mobile
Left Column	Date picker strip + time slot grid
Right Column (desktop)	Sticky summary card — remains visible while scrolling the slot grid
Footer Bar	Same sticky bar · Back + Continue (disabled until slot selected)

Components

Date Picker Strip

- Horizontal row of 7 date chips (today + next 6 days)
- Each chip: day name (Mon) + date number (3) · rounded-lg · 56px × 72px
- Selected date chip: bg #1D4ED8 · text white
- Other date chips: bg white · border #CBD5E1 · text #334155
- Dates with no available slots: text #94A3B8 · line-through · not clickable

Time Slot Grid

- Section heading — 'Available Slots for [Day, Date]' · H3
- 3-column grid of slot cards · each card: start time – end time · e.g., '08:00 – 10:00'
- Available slot: white card · border #CBD5E1 · body text showing '[N] of [Max] spots left'
- Nearly full slot (≥ 80% capacity): border #D97706 · warning badge pill 'Filling up' · bg #FFFBE8
- Fully booked slot: bg #F1F5F9 · text #94A3B8 · 'Fully Booked' label · not clickable · cursor not-allowed
- Selected slot: border #1D4ED8 2.5px · bg #F0F7FF · checkmark icon top-right

Summary Sidebar Card

- 'Booking Summary' · H3 · #0F172A
- Selected service row: service icon + name + price per kg
- Selected slot row: calendar icon + date + time range (shows '—' if not yet selected)
- Estimated weight input — pre-filled 0, stepper +/- buttons, 'kg' unit suffix — inline edit within sidebar
- Estimated total row — computed live: price_per_kg × weight · bold #1D4ED8
- Note — italic 12px: 'Final amount based on actual weight at pickup.'

States

- Loading — date strip shows shimmer; slot grid shows 9 shimmer slot placeholders
- No slots for selected date — empty state within slot grid: 'No slots available for this date. Try another day.'
- Slot selected — sidebar updates slot row and re-computes estimated total
- API error loading slots — error card: 'Could not load time slots. Please refresh.' with retry button

Micro-interactions

- Date chip selection: bg and text color swap over 150ms; previously selected chip returns to neutral
- Slot grid: fades in (opacity 0→1, 200ms) when a new date is selected, indicating fresh data
- Estimated total: number animates up or down when weight stepper changes (brief yellow highlight flash on the value for 300ms)
- 'Filling up' badge: subtle pulse animation (opacity 1→0.6→1 over 1.5s, repeating) to draw attention

Screen W-06 Booking Flow — Step 3: Confirm & Pay [Authenticated — Customer Role]

WashQ

Dashboard My Bookings Profile

✓ Select Service — ✓ Select Time Slot — 3 Confirm & Pay

Review Your Booking

Please confirm your booking details before proceeding to payment.

Service	Regular Wash	P45.00/kg
Time Slot	Monday, March 2 · 8:00 AM - 10:00 AM	
Estimated Weight	3 kg	
Estimated Total	P 135.00	

Special Instructions (Optional)

e.g., separate whites from colors...

Back PROCEED TO PAYMENT

Overview

Step 3 is the final confirmation screen before payment. The customer reviews all booking details, optionally adds special instructions, and submits. On submission the system locks the time slot, creates the booking record (status: PENDING), and redirects to the PayMongo checkout URL. The page includes a clear payment provider trust signal.

Layout Structure

Content Area	Max-width 720px · centered · single column
Confirmation Card	White card · radius 16px · padding 32px · full-width
Footer Bar	Sticky · Back + 'PROCEED TO PAYMENT' primary button

Components

Step Heading

- 'Review Your Booking' · H1
- Subtext — 'Please confirm your booking details before proceeding to payment.' · Body · #94A3B8

Booking Details Card

- Section label row: 'Service' label + selected service name + price/kg · separated by divider
- Time slot row: calendar icon + formatted date + time range
- Estimated weight row: scale icon + weight value + 'kg'
- Subtotal row: bold label 'Estimated Total' + computed amount in large #1D4ED8 bold text
- Note below subtotal: 'Final charge is based on actual weight measured at the shop.'

Special Instructions Field

- Label: 'Special Instructions' + '(Optional)' caption
- Textarea: 4 rows · placeholder 'e.g., separate whites from colors, handle with care...' · max 300 chars
- Character counter: right-aligned caption below textarea '0 / 300'

Payment Provider Badge

- Small horizontal row: lock icon (#16A34A) + 'Secured by PayMongo' · Caption · #94A3B8 · centered
- PayMongo logo SVG at small size (80px wide) beside lock row

Sticky Footer Bar

- 'Back' secondary button (left)
- 'PROCEED TO PAYMENT' primary button (right) — clicking triggers API call; button shows spinner during call

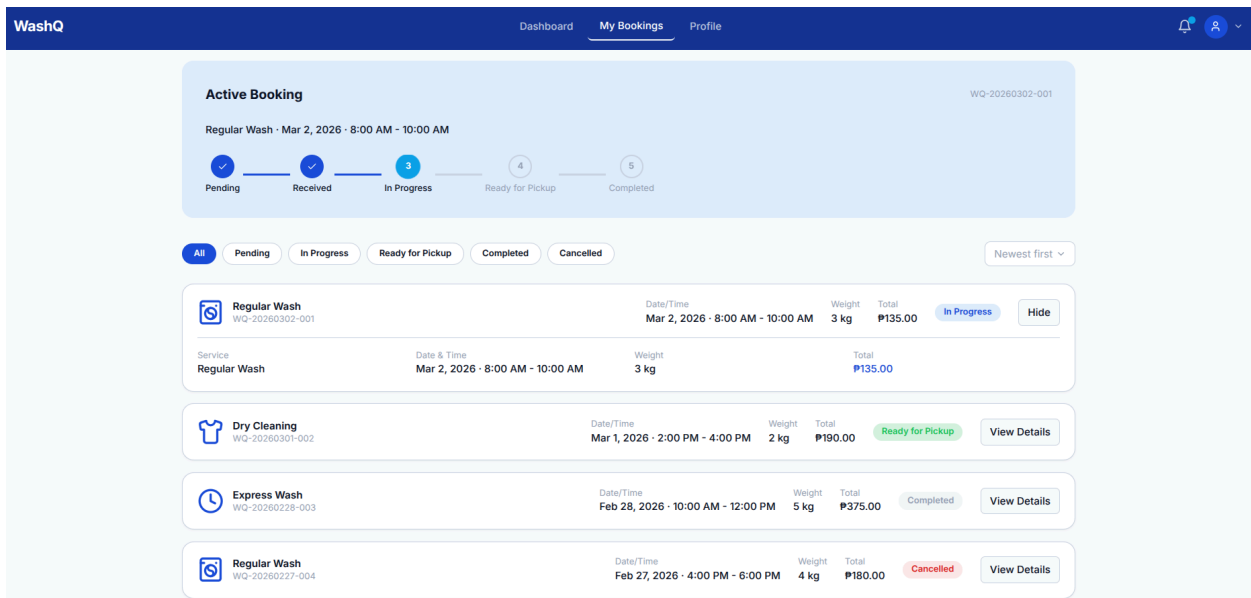
States

- Default — all details populated from previous steps; special instructions empty
- Submitting — 'PROCEED TO PAYMENT' shows spinner; both footer buttons disabled; a full-card overlay with spinner + 'Locking your slot...' message for 1–2 seconds
- Slot conflict error — if slot becomes fully booked between Step 2 and submission: error banner 'This slot was just filled. Please go back and select a different slot.' with 'Go Back' button
- General error — error banner with retry option

Micro-interactions

- Character counter turns #D97706 at 250 chars and #DC2626 at 300 chars
- On 'PROCEED TO PAYMENT' click: button scale(0.97) press effect then transitions to loading state
- Slot locking overlay: fade-in 200ms with a subtle animated lock icon (unlocked → locked keyframe)
- Redirect to PayMongo is seamless — no browser flash; page fades to white before location change

Screen W-07 Order History & Status Tracking [Authenticated — Customer Role]



Overview

This page lists all of the customer's bookings — past and present — with their current statuses. A prominent status tracker (progress stepper) is shown for the most recent active booking at the top. The full order list appears below as a filterable table/card list.

Layout Structure

Content Area	Max-width 1280px · centered · padding 32px 24px
Active Booking Tracker	Full-width card · shown only when there is at least one active (non-completed, non-cancelled) booking
Filter Bar	Full-width · status filter pills + date sort dropdown
Order List	Full-width · stacked booking cards on all breakpoints · pagination at bottom

Components

Active Booking Tracker Card

- Card: bg #DBEAFE · radius 16px · padding 24px
- Header: 'Active Booking' label (H3) + booking ID (Caption, right-aligned)
- Service name + time slot + date — Body 14px · #0F172A
- Status Progress Stepper — 5 steps horizontal: Pending → Received → In Progress → Ready for Pickup → Completed
 - Completed steps: filled #1D4ED8 circle with checkmark

- Current step: filled #0EA5E9 circle + pulsing ring animation
- Upcoming steps: hollow circle #CBD5E1
- Connector lines: filled left of current step, hollow right
- 'Cancelled' state replaces stepper with a full-width #FEE2E2 banner + 'This booking was cancelled.' text

Filter Bar

- Status filter pills — 'All', 'Pending', 'In Progress', 'Ready for Pickup', 'Completed', 'Cancelled' — pill-shaped toggle buttons; active pill: bg #1D4ED8 text white; inactive: bg white border #CBD5E1
- Sort dropdown — 'Newest first' / 'Oldest first' — right-aligned

Booking Card (per order)

- White card · radius 16px · shadow card · padding 20px
- Left: service icon (40px) + service name (H3) + booking ID (Caption)
- Center: slot date + time · estimated weight · total amount
- Right: status badge pill · 'View Details' button (secondary, small)
- Expandable: clicking 'View Details' expands the card inline to reveal full booking info + payment status + special instructions

Pagination

- 10 orders per page · prev/next arrow buttons + page number chips · centered

States

- Loading — skeleton cards (3–5 placeholders); filter bar loads immediately
- Empty — no orders: illustration + 'You haven't made any bookings yet.' + 'Book Now' CTA button
- Filtered empty — all filters produce no results: 'No bookings match this filter.' + 'Clear filters' link

Micro-interactions

- Active step pulse: pulsing ring on current status node (scale 1.0→1.4→1.0, opacity 1→0, 1.5s repeat)
- Card expansion: max-height animation from 0 to auto over 250ms ease-out; chevron icon rotates 90° on expand
- Filter pill selection: bg/text color transition 150ms; pill slides subtly downward 2px on active
- Status badge: on page load, badge fades in with a slight scale(0.9→1.0) over 200ms

Screen W-08 Profile Page [Authenticated — Customer Role]

Overview

The Profile page allows customers to view and edit their personal information and upload a profile picture. It uses a two-panel card layout: the avatar/upload section on the left and the editable form on the right. Changes are saved via a single 'Save Changes' button.

Layout Structure

Content Area	Max-width 960px · centered · padding 32px 24px
Page Heading	Full-width: 'My Profile' · H1 · breadcrumb: Home > Profile
Profile Card	Two-column card on desktop (avatar panel 30% + form 70%) · stacks on mobile
Avatar Panel	Left · bg #F1F5F9 · radius-left 16px · centered content
Form Panel	Right · bg white · radius-right 16px · padding 32px

Components

Avatar Panel

- Profile picture — circular, 120px diameter · object-fit cover
- Fallback avatar — initials (first + last initial) on #1D4ED8 background circle if no picture uploaded

- Upload button — 'CHANGE PHOTO' secondary button below avatar · clicking opens native file picker (accept: image/jpeg, image/png)
- File constraints note — Caption: 'JPEG or PNG · Max 2 MB'
- Upload progress indicator — circular progress ring around avatar (0–100%) on upload · replaces avatar temporarily with spinner overlay
- Remove photo link — small red link '#DC2626' below upload button · only visible if a photo is set

Profile Form

- Section heading — 'Personal Information' · H3
- Two-column name row: First Name + Last Name inputs
- Email input — disabled (read-only) · lock icon inside field · tooltip on hover: 'Email cannot be changed'
- Address textarea — 3 rows
- Contact Number input
- Divider
- Account section heading — 'Account Details' · H3
- Role badge — read-only pill: 'CUSTOMER' or 'STAFF' or 'ADMIN' in blue
- OAuth indicator — if Google account: Google icon + 'Linked with Google' · caption
- Change Password link — '#1D4ED8' text link · opens a modal (not in-line)
- 'SAVE CHANGES' primary button — right-aligned · disabled until a field is changed

Change Password Modal

- Elevated card modal · overlay bg rgba(0,0,0,0.4)
- Fields: Current Password · New Password · Confirm New Password
- Password strength indicator on new password
- Action buttons: 'Cancel' (secondary) + 'UPDATE PASSWORD' (primary)

States

- Default — form populated with current user data; 'Save Changes' disabled
- Editing — on any field change: 'Save Changes' button becomes enabled
- Uploading photo — progress ring animates; upload button disabled; 'Uploading...' caption
- Photo error — file too large or wrong type: inline error below button: 'File must be JPEG or PNG under 2 MB.'
- Save loading — 'Save Changes' shows spinner; form fields disabled
- Save success — toast notification: bg #DCFCE7 · 'Profile updated successfully.' · auto-dismiss 3s

- Save error — error banner inside form panel

Micro-interactions

- Avatar hover: slight overlay with camera icon and 'Change' label fades in on hover over the avatar circle
- Upload progress ring: SVG stroke-dashoffset animation from full to 0 as upload % increases
- Email field lock icon jiggles briefly (rotate $\pm 5^\circ \times 3$ over 300ms) on click attempt
- 'Save Changes' transitions from disabled to enabled over 150ms (opacity 40%→100%)
- Password modal: slides in from top + fades in 200ms; closing plays reverse animation

Screen W-09 Staff Dashboard [Authenticated — Staff / Admin Role]

WashQ

MAIN

Dashboard

Bookings

Services

Staff User

STAFF

Sign Out

Dashboard

24

Total Bookings Today

8

In Progress

5

Ready for Pickup

3

Pending Payment

Search bookings...

Status

Date Range

EXPORT CSV

BOOKING ID	CUSTOMER	SERVICE	SLOT	WEIGHT	AMOUNT	STATUS	ACTIONS
WQ-001	Juan Dela Cruz	Regular Wash	Mar 2, 8:00 AM	3 kg	₱135	In Progress	
WQ-002	Maria Santos	Dry Cleaning	Mar 2, 10:00 AM	2 kg	₱190	Pending	
WQ-003	Pedro Garcia	Express Wash	Mar 2, 2:00 PM	5 kg	₱375	Ready for Pickup	
WQ-004	Ana Reyes	Regular Wash	Mar 1, 4:00 PM	4 kg	₱180	Cancelled	
WQ-005	Jose Rizal	Dry Cleaning	Mar 1, 8:00 AM	1 kg	₱95	In Progress	

Overview

The Staff Dashboard uses the sidebar navigation layout. The main view is a booking management table showing all bookings across all customers, sortable and filterable. Staff can update booking statuses inline via a dropdown. A summary stats row at the top gives a quick operational overview.

Layout Structure

Page Background	bg #F8FAFC
-----------------	------------

Sidebar	240px fixed left · bg #0F172A · full viewport height · contains: WashQ logo, nav links, sign out
Main Area	Remaining width · padding 32px
Stats Row	4-column grid of stat cards · full main-area width
Filter/Search Bar	Full-width · below stats row · bg white · radius 12px · padding 16px
Bookings Table	Full-width card · white bg · radius 16px · shadow
Pagination	Below table · centered

Components

Sidebar Navigation

- WashQ logo — white wordmark, top of sidebar · padding 24px
- Nav section label — 'MAIN' Caption 12px #94A3B8 · uppercase tracking-wide
- Nav items: dashboard icon + 'Dashboard' (active: bg #1D4ED8 text white, radius 8px) · booking icon + 'Bookings' · service icon + 'Services' (Admin only)
- Bottom section: avatar + name + role chip · 'Sign Out' link with log-out icon
- Tablet: sidebar collapses to 60px icon-only rail; tooltip labels appear on icon hover

Stats Row

- 4 stat cards — white · radius 16px · shadow card · padding 20px
 - Total Bookings Today — count · calendar icon · #1D4ED8
 - In Progress — count · spinner icon · #D97706
 - Ready for Pickup — count · check icon · #16A34A
 - Pending Payment — count · credit card icon · #0EA5E9

Filter / Search Bar

- Search input — 'Search by customer name or booking ID...' · magnifier icon left · full-width or 40%
- Status filter dropdown — 'All Statuses' default · options: all booking statuses
- Date range picker — 'Today' default · dropdown: Today, This Week, This Month, Custom Range
- 'EXPORT CSV' tertiary button — right-aligned · outline style · downloads filtered results

Bookings Table

- Table header row: bg #F8FAFC · font 12px SemiBold uppercase #94A3B8 · columns: Booking ID | Customer | Service | Slot | Est. Weight | Amount | Status | Actions
- Table rows — alternating white / #F8FAFC · 56px row height · Body 14px
- Status column: renders appropriate badge pill per booking status
- Actions column: dropdown button ':' (vertical ellipsis) · on click: dropdown card with 'Update Status', 'View Details', and (for Admin) 'Cancel Booking'
- 'Update Status' action: opens an inline dropdown within the table row — shows only the next valid status transition; selecting confirms immediately with a micro-toast
- Row click: expands to show full booking details inline below row (accordion pattern)

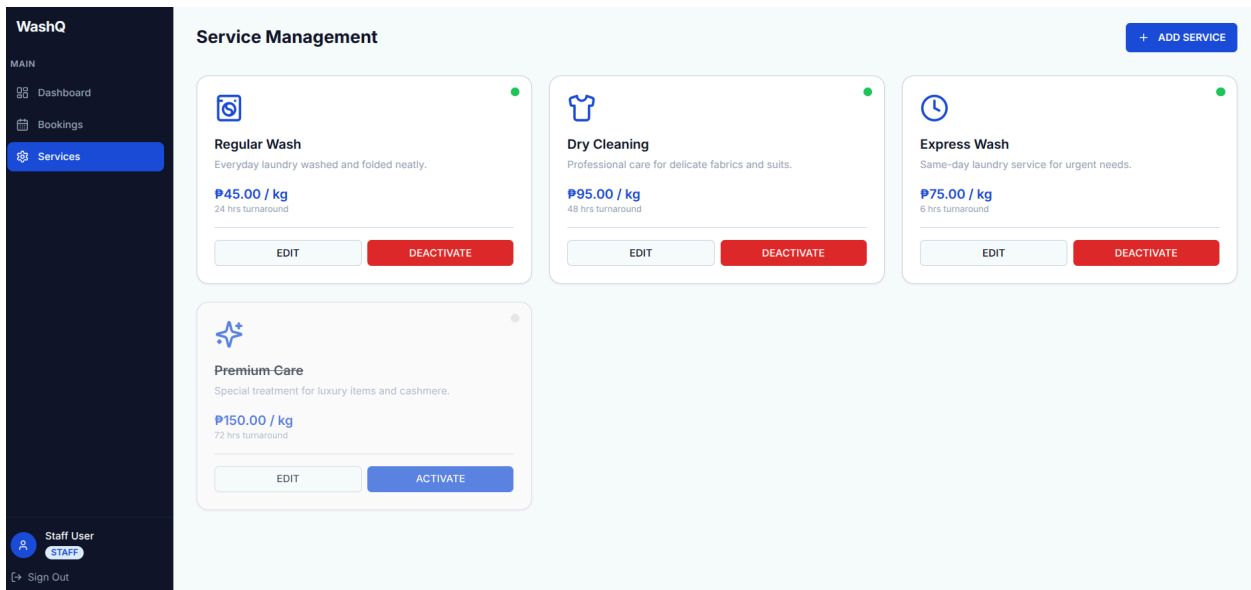
States

- Loading — table shows 8 skeleton rows; stats show shimmer numbers
- Empty (no bookings for filter) — table body replaced by centered: 'No bookings found for the selected filters.'
- Status update loading — the actions ':' button shows a spinner during the API call; row dims to 60% opacity
- Status update success — row's status badge updates instantly (optimistic update) then confirms; a micro-toast appears bottom-right: 'Status updated to In Progress' for 2.5s
- Status update error — badge reverts to previous value; error toast appears

Micro-interactions

- Sidebar active link: bg fills from left (width 0→100% in 150ms) on navigation
- Stats cards: numbers count up on page load from 0 to actual value over 500ms ease-out
- Table row hover: bg transitions to #F0F7FF over 150ms
- Actions dropdown: slides in from right side of the trigger button with 150ms ease-out
- Status badge change: old badge fades out + new badge fades in over 200ms; no layout shift
- Micro-toast (bottom-right): slides up from bottom + fades in on appear; slides down + fades out on dismiss after 2.5s

Screen W-10 Admin — Service Management [Authenticated — Admin Role Only]



Overview

The Service Management page is accessible only to Admin users. It uses the same sidebar layout as the Staff Dashboard. Services are shown as a card grid with inline edit and deactivate actions. A floating '+' button opens a modal to add a new service.

Layout Structure

Sidebar	Same as Staff Dashboard
Main Area	Max-width within main area · padding 32px
Page Header	Full-width row: 'Service Management' H1 (left) + 'ADD SERVICE' primary button (right)
Service Grid	3-column on desktop · 2-col tablet · 1-col mobile · gap 24px
Add/Edit Modal	Centered modal overlay · max-width 540px · elevated card

Components

Service Card (Admin View)

- White card · radius 16px · shadow card · padding 24px
- Status indicator dot: top-right corner — green (active) or grey (inactive/deactivated)
- Service icon · service name H3 · description Body (truncated 2 lines)
- Price — '₹ [X].00 / kg' · 18px SemiBold #1D4ED8
- Duration — '~X hours' caption

- Card footer: divider line + two action buttons side by side:
 - 'EDIT' — secondary button, full-width 50%
 - 'DEACTIVATE' or 'ACTIVATE' — danger or success button, full-width 50%; toggle based on current active state

Add / Edit Service Modal

- Modal header: 'Add New Service' or 'Edit Service' · H2 · X close button (top-right)
- Service Name input — required
- Description textarea — 3 rows · optional
- Price per kg input — number type · ₪ prefix adornment · step 0.50 · required
- Estimated Duration input — number type · 'hours' suffix adornment · required
- Active toggle switch — label 'Service Active' · right side · on by default for new services
- Modal footer: 'Cancel' secondary button + 'SAVE SERVICE' primary button

Deactivate Confirmation Dialog

- Small confirmation modal: 'Deactivate [Service Name]?' · H3
- Body: 'This service will no longer appear to customers. Existing bookings are not affected.'
- Two buttons: 'Cancel' + 'DEACTIVATE' (danger button #DC2626)

States

- Loading — skeleton service cards with shimmer
- Empty — no services: centered 'No services found.' + 'ADD SERVICE' CTA
- Inactive card visual: card opacity 70% · service name has strikethrough · status dot grey · 'ACTIVATE' button replaces 'DEACTIVATE'
- Modal saving — 'SAVE SERVICE' shows spinner; inputs disabled
- Save success — modal closes; service card appears/updates with a brief green highlight flash (bg #DCFCE7 on card for 1s then fades to white); success toast bottom-right
- Save error — inline error banner inside modal; modal stays open

Micro-interactions

- 'ADD SERVICE' button: icon rotates 45° (+ to ×) when modal opens — optional pattern if button doubles as toggle
- New service card: animates in from scale(0.95) + opacity 0 → scale(1) + opacity 1 over 250ms when added
- Deactivate: card opacity transitions to 70% over 300ms; 'Deactivated' toast bottom-right

- Reactivate: card opacity transitions back to 100% over 300ms
- Toggle switch (Active): smooth thumb slide over 200ms with bg color transition from #94A3B8 to #16A34A
- Edit modal: field values pre-populate with a subtle fade-in over 100ms to signal the pre-fill

7.3 Web Screens Summary

The table below provides a quick reference of all ten web application screens.

ID	Screen Name	Access Role	Route	Nav Layout
W-01	Login	Public	/login	None (split panel)
W-02	Registration	Public	/register	None (split panel)
W-03	Customer Dashboard	Customer	/dashboard	Top Navbar
W-04	Booking — Select Service	Customer	/booking/service	Top Navbar
W-05	Booking — Select Slot	Customer	/booking/slot	Top Navbar
W-06	Booking — Confirm & Pay	Customer	/booking/confirm	Top Navbar
W-07	Order History & Tracking	Customer	/orders	Top Navbar
W-08	Profile	Customer	/profile	Top Navbar
W-09	Staff Dashboard	Staff / Admin	/staff/dashboard	Sidebar
W-10	Service Management	Admin Only	/admin/services	Sidebar

7.4 Mobile Application Design System

The Android mobile application shares the WashQ brand identity and color palette defined in Section 7.1 but applies a separate set of layout rules, spacing tokens, and component patterns suited to a native Android environment. The app is built with Kotlin and XML-based UI layouts targeting API Level 34 (Android 14) with a minimum of API Level 26 (Android 8.0).

Where values differ from the web design system, the mobile values defined in this section take precedence for Android screens.

7.4.1 Screen Canvas & Density

Target Canvas	360 × 800 dp (compact phone) — all screens designed at this base size
Density Bucket	xxhdpi (480 dpi) as primary export target; assets provided at 1×, 2×, 3× densities
Status Bar	Transparent or bg #0F172A · light icon tint · edge-to-edge layout
Navigation Bar	System gesture navigation (Android 10+) preferred · legacy 3-button bar handled with window insets
Safe Area	Top inset: status bar height (~24dp) · Bottom inset: gesture/nav bar height (~16–32dp)

7.4.2 Color Palette — Mobile

The mobile palette mirrors the web palette with two additions for native Android surfaces.

Primary #1D4ED8	FABs, primary buttons, active bottom nav icon, filled chips
Primary Dark #1E3A8A	Top app bar background, status bar tint
Primary Light #DBEAFE	Card highlights, selected chip bg, shimmer base
Accent #0EA5E9	Icon tints, status indicator dots, progress rings
Success #16A34A	Confirmed status, success snackbar icon
Warning #D97706	Filling-up slot chip, pending badge
Error #DC2626	Inline validation, error snackbar
Surface Dark #0F172A	Bottom navigation bar bg, drawer header bg [Mobile addition]
Background #F8FAFC	Screen background (window background)
Surface #FFFFFF	Cards, bottom sheets, dialogs
Surface Variant #F1F5F9	Input field backgrounds, dividers
On-Surface #334155	Body text, secondary labels
On-Surface Muted #94A3B8	Placeholder text, disabled labels, captions
On-Surface Strong #0F172A	Headings, bold labels

7.4.3 Typography — Mobile

Font Family	Inter — loaded as a custom font via res/font/ in the Android project
Display / Hero	28sp / Bold / #0F172A — screen titles on auth pages
Title Large	22sp / SemiBold / #0F172A — toolbar titles, card headers
Title Medium	18sp / SemiBold / #334155 — section headings within screens
Body Large	16sp / Regular / #334155 — primary body text, input values
Body Medium	14sp / Regular / #334155 — secondary body, list item subtitles
Label / Caption	12sp / Regular / #94A3B8 — timestamps, helper text, badge labels

Button	14sp / SemiBold / ALL CAPS letter-spacing 1.25 — all button labels
Overline	11sp / SemiBold / UPPERCASE / #94A3B8 — section labels, chip text

7.4.4 Spacing & Touch Targets

Base Unit	4dp — all spacing is a multiple of 4dp
Screen Horizontal Margin	16dp on both sides
Section Vertical Spacing	24dp between major content sections
Card Internal Padding	16dp (compact) or 20dp (standard)
Minimum Touch Target	48 × 48dp — all interactive elements meet this minimum per Material guidelines
List Item Height	56dp (single-line) · 72dp (two-line) · 88dp (three-line)
Bottom Navigation Height	56dp (icon + label)
Top App Bar Height	56dp

7.4.5 Component Tokens — Mobile

Primary Button	bg #1D4ED8 · text white · radius 8dp · padding 12dp 24dp · min-width 88dp · min-height 48dp · ripple white/20%
Secondary Button (Outlined)	border 1.5dp #1D4ED8 · text #1D4ED8 · bg transparent · radius 8dp · ripple #DBEAFE
Danger Button	bg #DC2626 · text white · radius 8dp — destructive actions only
Text Button	#1D4ED8 text · no border/bg · ripple #DBEAFE · used for inline actions and links
FAB (Floating Action Button)	bg #1D4ED8 · white icon · radius 16dp · shadow elevation 6dp · size 56×56dp
Card	bg white · radius 12dp · elevation 1dp (shadow 0 1px 3px rgba(0,0,0,0.1)) · padding 16dp

Elevated Card	elevation 4dp (shadow 0 4px 12px rgba(0,0,0,0.1)) — modal bottom sheets, dialogs
Input Field (Outlined)	border #CBD5E1 1dp · radius 8dp · padding 14dp 12dp · focus border #1D4ED8 2dp · error border #DC2626
Input Field (Filled)	bg #F1F5F9 · radius 8dp top · bottom border only · focus bottom border #1D4ED8 2dp
Bottom Navigation	bg #0F172A · active icon+label #1D4ED8 or white · inactive icon #94A3B8 · 56dp height
Bottom Sheet	bg white · radius 16dp top-left + top-right · drag handle 40×4dp centered at top
Snackbar	bg #0F172A · text white · action text #0EA5E9 · radius 8dp · 8dp from screen bottom edge
Badge — Pending	bg #FEF3C7 · text #D97706 · radius 9999dp · 11sp SemiBold · padding 3dp 8dp
Badge — In Progress	bg #DBEAFE · text #1D4ED8 · radius 9999dp
Badge — Ready for Pickup	bg #DCFCE7 · text #16A34A · radius 9999dp
Badge — Completed	bg #F1F5F9 · text #334155 · radius 9999dp
Badge — Cancelled	bg #FEE2E2 · text #DC2626 · radius 9999dp
Chip (Filter)	radius 9999dp · 14sp · padding 8dp 14dp · selected: bg #1D4ED8 text white · unselected: bg #F1F5F9 text #334155 border #CBD5E1
Divider	1dp · #F1F5F9 · full-width or inset 16dp from left
Progress Ring	Stroke width 3dp · color #1D4ED8 · indeterminate or determinate

7.4.6 Navigation Architecture

Auth Flow	Login Screen → Registration Screen (no persistent navigation chrome)
Post-Login — Customer	Bottom Navigation Bar with 3 tabs: Home, My Bookings, Profile
Post-Login — Staff/Admin	Bottom Navigation Bar with 2 tabs: Bookings, Profile (staff sees no service tab)

Bottom Nav Tabs — Customer	Home (house icon) · My Bookings (list icon) · Profile (person icon)
Booking Wizard	Launched from Home tab; pushed onto the back stack as a 3-screen flow (Select Service → Select Slot → Confirm); toolbar Back arrow navigates between steps
Deep Screens	Order Detail, Settings pushed onto back stack from their parent tabs
Transitions	Standard slide-in-right for forward navigation; slide-out-right for back; fade for tab switches
JWT Storage	EncryptedSharedPreferences — token persists across app restarts; invalid/expired token redirects to Login

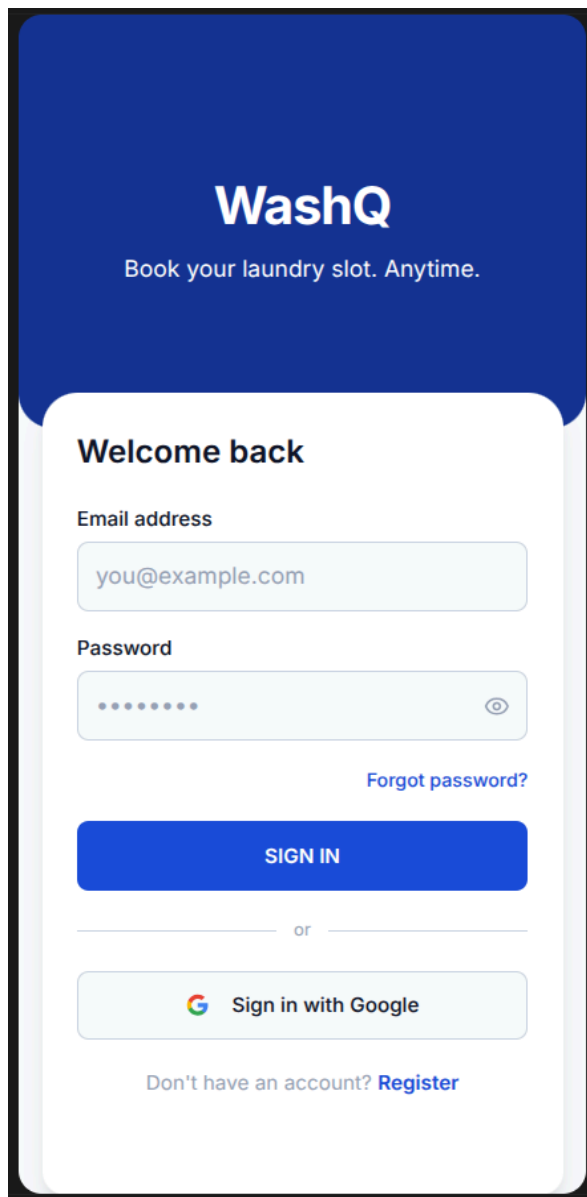
7.4.7 Android-Specific Patterns

- Edge-to-edge layout: `WindowCompat.setDecorFitsSystemWindows(window, false)` applied in `BaseActivity`. All screens handle top/bottom window insets using `ViewCompat.setOnApplyWindowInsetsListener`.
- Ripple feedback: all interactive surfaces use the system ripple drawable with bounded or unbounded ripple per Material guidelines.
- Loading skeletons: shimmer effect using a custom `ShimmerFrameLayout` wrapping placeholder views — matches the exact shape of the real content beneath.
- Pull-to-refresh: `SwipeRefreshLayout` on all list/feed screens; refresh color `#1D4ED8`.
- Keyboard behavior: `windowSoftInputMode` set to `adjustResize` on screens with forms so inputs are never hidden by the soft keyboard.
- Empty states: each list screen has a dedicated empty state layout (illustration SVG + heading + body + CTA button) shown when the data set is empty.
- Error states: network/API errors show a persistent error banner (bg `#FEE2E2`, error icon, message, 'Retry' text button) replacing the content area rather than a blocking dialog.

7.5 Mobile Application Screens

The following seven screens cover the complete mobile application for Android. Each screen is described for a 360×800dp canvas. Screen IDs are prefixed with 'M-' to distinguish from web screens.

Screen M-01 Login Screen [Public — All Users]



The image shows a mobile application login screen for 'WashQ'. The top half of the screen has a dark blue background with the 'WashQ' logo in white and the tagline 'Book your laundry slot. Anytime.' below it. The bottom half is a white rounded rectangle containing the login form. The form starts with the text 'Welcome back'. It has two input fields: 'Email address' with the placeholder 'you@example.com' and 'Password' with a masked password '.....' and an eye icon for toggling visibility. Below the password field is a blue link 'Forgot password?'. A blue 'SIGN IN' button is positioned below the password field. A horizontal line with the word 'or' in the center separates the 'SIGN IN' button from the 'Sign in with Google' button, which features the Google logo. At the bottom of the form is a link 'Don't have an account? Register'.

Overview

The Login screen is the app's entry point. It uses a full-screen single-column layout — no split panel unlike the web version. The top half is a brand hero area; the bottom half is the form. The keyboard pushes content up via `adjustResize`.

Layout Structure

Screen Background	bg #F8FAFC · edge-to-edge
Top Hero Area	~40% screen height · bg #1E3A8A · WashQ logo centered (white, 48dp) · tagline 'Book your laundry slot.' Inter 16sp white · bottom rounded corners radius 24dp
Form Container	White bg · rounded top corners 24dp · overlaps hero bottom by 24dp (card lift effect) · padding 24dp · scroll enabled if keyboard shown
Form Width	Full screen width minus 32dp horizontal margins

Components

- Status bar: transparent, light icons over #1E3A8A hero
- WashQ logo: white SVG wordmark, 140dp wide, centered in hero
- Tagline: 'Book your laundry slot. Anytime.' · 16sp regular white · centered
- Form heading: 'Welcome back' · 22sp SemiBold #0F172A · top of form
- Email outlined input: label 'Email address' · type textEmailAddress · keyboard: email · full width
- Password outlined input: label 'Password' · type textPassword · end icon: show/hide eye toggle · full width
- 'Forgot password?' — text button right-aligned · 13sp #1D4ED8 · 8dp below password field
- 'SIGN IN' primary button: full width · 48dp height · bg #1D4ED8 · radius 8dp · 16dp top margin
- Divider row: thin 1dp line #F1F5F9 on each side of 'or' caption text · 24dp vertical spacing
- 'Sign in with Google' outlined button: full width · Google G icon 20dp left · border #CBD5E1
- Registration link: 'Don't have an account?' body + 'Register' in #1D4ED8 SemiBold · centered · 16dp top margin

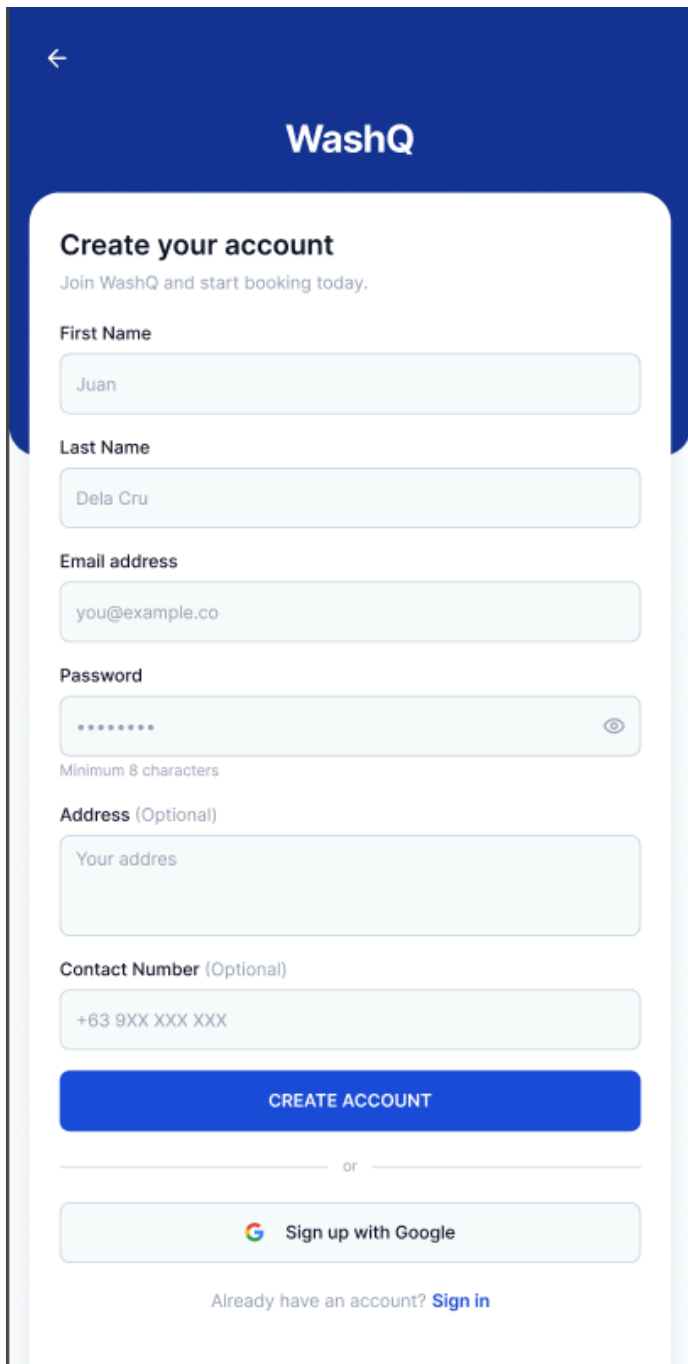
States

- Default — inputs empty, SIGN IN button enabled (no pre-validation)
- Loading — SIGN IN button shows circular progress indicator (white, 20dp) replacing label; inputs disabled; keyboard dismissed
- Error — Snackbar bottom: bg #0F172A text 'Invalid email or password.' action 'DISMISS'; input borders turn #DC2626 with error caption below each
- Validation — inline error captions on lost focus: red 12sp below field, border #DC2626

Micro-interactions & Android Patterns

- Hero area: subtle parallax — scrolling the form up slightly reveals more of the hero gradient (20dp range)
- Password toggle: eye icon animates between filled/outline variant with 150ms alpha transition
- SIGN IN ripple: bounded white ripple on button press before loading state
- Form card lifts into view with a 300ms slide-up + fade-in on initial screen entry
- Keyboard shown: form scrolls to keep focused input visible above keyboard; hero area shrinks proportionally

Screen M-02 Registration Screen [Public — New Users]



The image shows a mobile app registration screen for 'WashQ'. At the top, there's a dark blue header with a white back arrow on the left and the 'WashQ' logo in white. Below the header is a white rounded rectangle containing the registration form. The form starts with the title 'Create your account' and a subtitle 'Join WashQ and start booking today.' The form fields are: 'First Name' (with 'Juan' entered), 'Last Name' (with 'Dela Cru' entered), 'Email address' (with 'you@example.co' entered), 'Password' (with masked characters and a toggle icon, with a note 'Minimum 8 characters' below it), 'Address (Optional)' (with 'Your address' entered), and 'Contact Number (Optional)' (with '+63 9XX XXX XXX' entered). A blue 'CREATE ACCOUNT' button is below the fields. Below the button is a horizontal line with 'or' in the center. Underneath is a button with the Google logo and 'Sign up with Google'. At the bottom, there's a link: 'Already have an account? [Sign in](#)'.

←

WashQ

Create your account

Join WashQ and start booking today.

First Name

Juan

Last Name

Dela Cru

Email address

you@example.co

Password

.....

Minimum 8 characters

Address (Optional)

Your address

Contact Number (Optional)

+63 9XX XXX XXX

CREATE ACCOUNT

or

 Sign up with Google

Already have an account? [Sign in](#)

Overview

The Registration screen shares the same hero+form layout as Login. The form is longer and scrollable. A back arrow in the top-left of the hero returns to the Login screen. On successful registration a welcome snackbar fires and the user is navigated to the Home dashboard.

Layout Structure

Screen Background	bg #F8FAFC
Top Hero	~30% height (shorter than Login to give more room to form) · bg #1E3A8A · back arrow IconButton top-left (white, 24dp) · WashQ logo centered
Form Container	White card · rounded top 24dp · overlapping hero · vertically scrollable · padding 24dp 16dp

Components

- Back arrow: top-left of hero · navigates to Login (no back-stack push; replace)
- Form heading: 'Create your account' · 22sp SemiBold
- Form subtext: 'Join WashQ and start booking today.' · 14sp #94A3B8
- First Name outlined input · full width
- Last Name outlined input · full width · 8dp below First Name
- Email outlined input · textEmailAddress keyboard
- Password outlined input · show/hide toggle · helper text: 'Minimum 8 characters'
- Password strength bar: full-width horizontal bar below password input · 4dp height · radius 9999dp · color transitions: #DC2626 (weak) → #D97706 (moderate) → #16A34A (strong) · animated width
- Address outlined input · textMultiLine · minLines 2 · inputType textMultiLine · optional
- Contact Number outlined input · phone keyboard · placeholder '+63 9XX XXX XXXX' · optional
- 'CREATE ACCOUNT' primary button · full width · 48dp height
- Divider 'or'
- 'Sign up with Google' outlined button · full width · Google G icon
- 'Already have an account? Sign in' link · centered · 'Sign in' in #1D4ED8

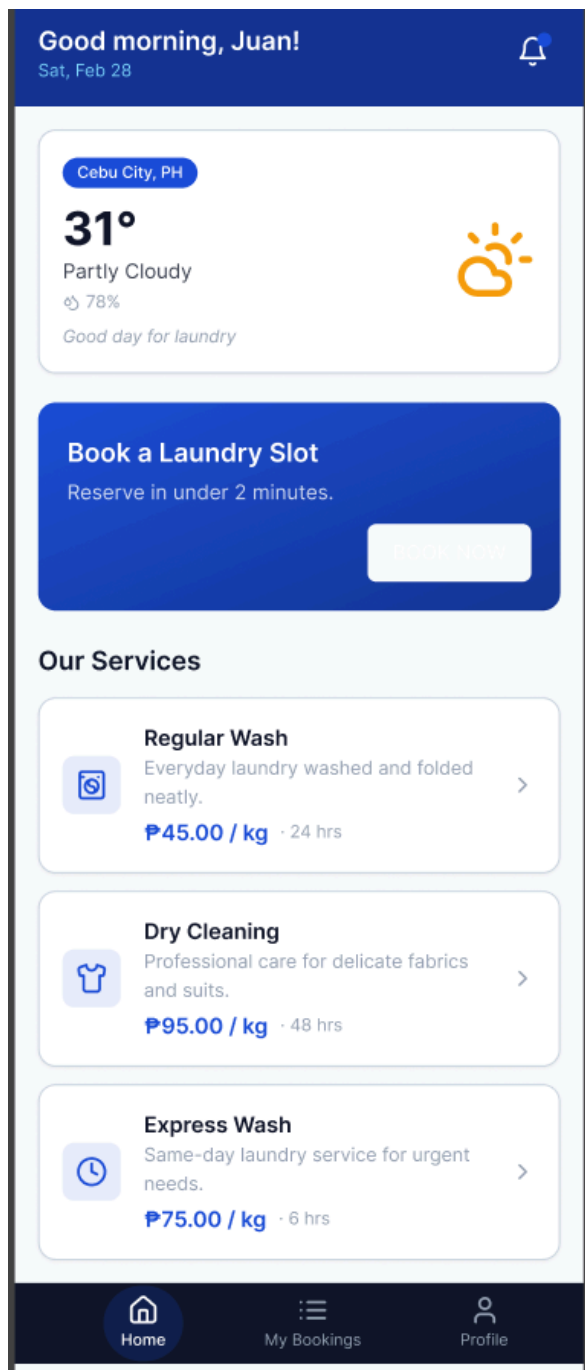
States

- Default — all fields empty
- Inline validation — each required field shows error on lost focus
- Password strength — bar updates on every keystroke
- Loading — CREATE ACCOUNT shows spinner; all inputs and buttons disabled
- Duplicate email error — snackbar: 'This email is already registered.' with 'SIGN IN' action that navigates to Login
- Success — snackbar bottom: bg #0F172A icon check green 'Account created! Check your email.' · auto-dismiss 4s · navigate to Home

Micro-interactions & Android Patterns

- Password strength bar: animated width via ValueAnimator (300ms) + color via ArgbEvaluator on each keypress
- Form scroll: NestedScrollView containing all fields; adjustResize so keyboard never covers active input
- CREATE ACCOUNT: ripple on tap → loading state → success snackbar sequence
- Transition to Home: ActivityOptions.makeCustomAnimation slide-up fade

Screen M-03 Home / Dashboard Screen [Authenticated – Customer Role]



Overview

The Home screen is the customer's main screen after login. It uses a `NestedScrollView` with a top app bar that collapses as the user scrolls. Content is organized in vertically stacked sections: weather widget, quick-book CTA, service catalog. The bottom navigation bar provides tab switching between Home, My Bookings, and Profile.

Layout Structure

Screen Background	bg #F8FAFC
Top App Bar	56dp · bg #1E3A8A · CollapsingToolbarLayout: expanded shows 'Good morning, [Name]!' title + date subtitle; collapsed shows 'WashQ' toolbar title · notification bell icon right
Bottom Navigation Bar	56dp · bg #0F172A · 3 tabs: Home (active), My Bookings, Profile
Content	NestedScrollView · padding 16dp horizontal · sections stacked with 20dp gaps
FAB	Not present on Home — FAB is on the Booking flow screens

Components

Collapsing Top App Bar (Expanded State)

- bg #1E3A8A · extended height ~140dp
- Greeting: 'Good morning, [Name]!' · 22sp SemiBold white
- Date: 'Saturday, February 28, 2026' · 13sp #93C5FD (light blue)
- Notification bell icon: white 24dp · badge dot #0EA5E9 if unread

Weather Widget Card

- White card · radius 12dp · elevation 1dp · padding 16dp
- Two-column row: LEFT — temperature 32sp Bold #0F172A, condition 14sp #334155, humidity icon+caption 12sp #94A3B8. RIGHT — weather icon/illustration 64dp
- Location chip top: 'Cebu City, PH' · sky-blue #0EA5E9 filled chip · 11sp
- Advisory note bottom: italic 12sp #94A3B8 — 'Good day for laundry!' or 'Consider indoor drying.'
- Loading state: shimmer placeholder matching exact card dimensions

Quick Book CTA Card

- Card bg gradient #1D4ED8 → #1E3A8A · radius 12dp · padding 20dp
- 'Book a Laundry Slot' · 18sp SemiBold white
- Subtitle: 'Reserve in under 2 minutes.' · 14sp white/80%
- 'BOOK NOW' outlined button: white border + white text · radius 8dp · right-aligned · 48dp height

Services Section

- Section header row: 'Our Services' 18sp SemiBold #0F172A left · no 'See all' link (all services shown)
- Vertical list of service cards (not grid — single column on mobile)
- Service card (white, radius 12dp, elevation 1dp, padding 16dp): horizontal layout — service icon 40dp left (#1D4ED8) · RIGHT: service name 16sp SemiBold, description 13sp #94A3B8 2-line max, price #1D4ED8 16sp SemiBold, duration 12sp #94A3B8 · far-right: chevron icon #94A3B8
- Tapping any service card navigates to Booking Step 1 with that service pre-selected

Bottom Navigation Bar

- bg #0F172A · 3 tabs with icon + label
- Home: house icon · label 'Home' — active: icon+label #FFFFFF, indicator pill #1D4ED8
- My Bookings: list icon · label 'Bookings' — inactive: icon+label #94A3B8
- Profile: person icon · label 'Profile' — inactive: icon+label #94A3B8
- Active indicator: rounded rectangle bg #1D4ED8/20% behind active icon (Material 3 pattern)

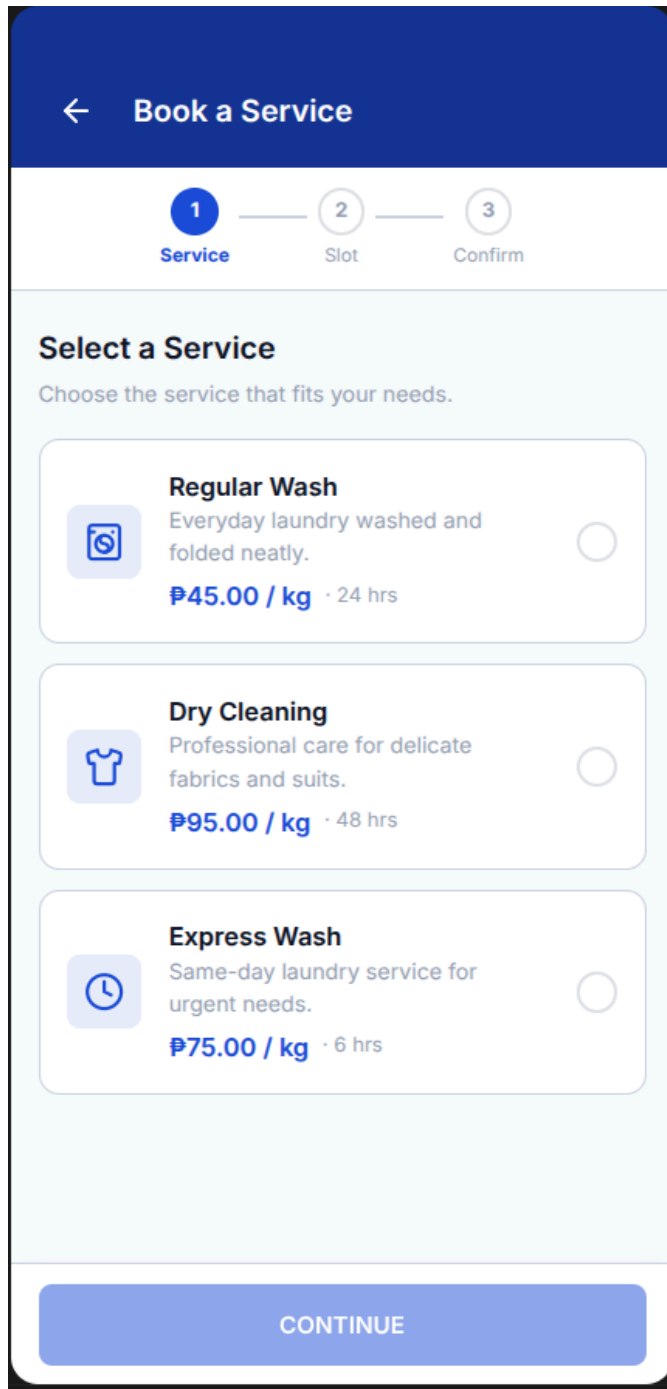
States

- Loading — CollapsingToolbar shows skeleton for greeting; weather card shimmer; service list shows 3 shimmer cards
- Weather error — weather card: cloud-off icon + 'Weather unavailable' + 'Retry' text button
- No services — services section: empty state layout (icon + 'No services available.')

Micro-interactions & Android Patterns

- CollapsingToolbarLayout: title crossfades from expanded greeting to collapsed 'WashQ' as user scrolls (parallax factor 0.5 on hero background)
- Service card press: elevation lifts from 1dp → 4dp on press (300ms); ripple effect on card surface
- Bottom nav tab switch: icon scale 1.0→1.2→1.0 on selection (200ms spring); content fades in (150ms)
- Weather temperature: counts up from 0 on first load using ValueAnimator (400ms ease-out)
- Pull-to-refresh: SwipeRefreshLayout with #1D4ED8 spinner refreshes weather + services

Screen M-04 Booking Flow — Step 1: Select Service [Authenticated — Customer Role]



Overview

Launched from the Home screen, the booking wizard uses a dedicated Activity with its own back stack. A horizontal stepper below the toolbar tracks progress. Step 1 shows all available services as a full-width vertical list of selectable cards. The 'Continue' button in the bottom sticky bar is disabled until a service is selected.

Layout Structure

Screen Background	bg #F8FAFC
Top App Bar	56dp · bg #1E3A8A · back arrow left · title 'Book a Service' · no overflow menu
Step Indicator	Below toolbar · bg white · 3 steps horizontal · step 1 active · full-width · 56dp height
Content	ScrollView · padding 16dp · service card list · bottom padding 80dp to clear sticky bar
Sticky Bottom Bar	bg white · border-top #F1F5F9 · elevation 8dp · padding 12dp 16dp · 'CONTINUE' button full-width

Components

Step Indicator Bar

- Three steps connected by lines: step number circles 28dp diameter
- Active step (1): filled circle #1D4ED8 · white number · label 'Service' below · 12sp #1D4ED8 bold
- Inactive steps (2, 3): hollow circle border #CBD5E1 · grey number #94A3B8 · labels 'Slot', 'Confirm'
- Connecting lines: 1dp · #CBD5E1 — fills to #1D4ED8 for completed steps

Service Selection Cards

- Full-width cards (no horizontal grid on mobile) · white · radius 12dp · elevation 1dp · padding 16dp
- Layout: service icon 44dp #1D4ED8 left · text block center (name 16sp SemiBold, description 13sp #94A3B8, price 16sp SemiBold #1D4ED8, duration 12sp #94A3B8) · selection radio indicator right
- Unselected: card border #F1F5F9 1dp · radio button outline #CBD5E1
- Selected: card border #1D4ED8 2dp · card bg #F0F7FF · radio button filled #1D4ED8 · checkmark replaces radio circle
- Gap between cards: 12dp

Sticky Bottom Bar

- 'CONTINUE' primary button: full-width · 48dp · bg #1D4ED8 · disabled state: bg #CBD5E1 text #94A3B8

States

- Default — no service selected; CONTINUE disabled (greyed)
- Selected — one card highlighted; CONTINUE enabled and #1D4ED8
- Loading — 4 shimmer card placeholders; CONTINUE disabled

Micro-interactions & Android Patterns

- Card selection: border + bg color animated with ValueAnimator (150ms); checkmark icon fades in from alpha 0 + scaleX/Y 0.7 → 1.0
- CONTINUE button: color transition from grey → #1D4ED8 over 200ms on selection
- Back arrow: pops the booking Activity and returns to Home (with shared element if service icon is used)

Screen M-05 Booking Flow — Step 2: Select Time Slot [Authenticated — Customer Role]

← Select Time Slot

Service Slot Confirm

MON 3 TUE 4 WED 5 THU 6 FRI 7 SA 8

Available on Monday, March 3

08:00 – 10:00
3 spots left

10:00 – 12:00 Filling up
1 spots left

12:00 – 14:00 Fully Booked

14:00 – 16:00
4 spots left

16:00 – 18:00
2 spots left

Regular Wash ₱45.00 / kg

– 5 + kg ≈ ₱225.00

CONTINUE

Overview

Step 2 is a two-section scrollable screen: a horizontal date chip row followed by a vertical list of time slot cards for the selected date. A mini summary bar at the bottom (above the CONTINUE button) shows the selected service so the user maintains context. There is no sidebar — the summary is inline at the bottom.

Layout Structure

Top App Bar	56dp · bg #1E3A8A · back arrow · title 'Select Time Slot'
Step Indicator	Step 2 active · same component as Step 1
Date Chip Row	Horizontal HorizontalScrollView · padding 16dp · 7 date chips · no scroll indicator
Slot List	Vertical RecyclerView (LinearLayoutManager) · padding 16dp · clips to padding false
Summary Bar	bg #DBEAFE · padding 12dp 16dp · selected service summary row
Sticky Bottom Bar	bg white · CONTINUE button (disabled until slot selected) + estimated total

Components

Date Chip Row

- 7 scrollable chips, each 56×72dp · radius 12dp
- Each chip: day abbreviation (Mon) 11sp uppercase #94A3B8 top · date number 20sp SemiBold center · available dot 6dp #16A34A below number (hidden if no slots)
- Selected chip: bg #1D4ED8 · day label white/70% · date number white bold · dot white
- Unavailable chip: bg #F1F5F9 · text #CBD5E1 · no dot · not clickable

Time Slot Cards

- Section label: 'Available on [Day, Date]' · 16sp SemiBold #0F172A · 16dp top margin
- Each slot card: white · radius 12dp · elevation 1dp · padding 14dp 16dp · full-width
- Slot card layout: time range '08:00 – 10:00' 16sp SemiBold left · 'X spots left' caption 12sp #94A3B8 below time · selection radio right
- Nearly full slot: 'Filling up' chip #FEF3C7/#D97706 after the time range · card bg #FFFBEB · border #D97706 1dp
- Fully booked: bg #F1F5F9 · time text #94A3B8 · 'Fully Booked' chip #FEE2E2/#DC2626 · not clickable · 50% alpha
- Selected: border #1D4ED8 2dp · bg #F0F7FF · radio filled #1D4ED8

Summary Bar

- bg #DBEAFE · padding 12dp 16dp · single-line row

- Service icon 24dp · service name 14sp SemiBold #0F172A · price 14sp #1D4ED8 SemiBold right-aligned

Sticky Bottom Bar

- Estimated weight stepper inline: '-' icon button · weight value 16sp · '+' icon button · 'kg' label
- Estimated total: '≈ ₱ 0.00' 14sp · updates live as weight changes
- 'CONTINUE' primary button · full-width · 48dp · disabled until slot selected

States

- Loading — date chips shimmer; slot list shows 5 shimmer rows
- No slots for date — slot list area: empty state 'No slots available for this date.' with suggestion to try another day
- Slot selected — CONTINUE enables; summary bar updates slot time
- API error — error banner replaces slot list with retry button

Micro-interactions & Android Patterns

- Date chip selection: bg and text animated (150ms); slot list fades in (alpha 0→1, 200ms) when date changes — signals fresh data
- 'Filling up' chip: subtle pulse animation on the chip (alpha 1→0.5→1 repeating every 2s)
- Weight stepper: estimated total updates with a brief #DBEAFE background flash on the total value (300ms fade)
- RecyclerView slot list: items animate in with a staggered slide-up (30dp) + fade, 50ms delay per item

Screen M-06 Booking Flow — Step 3: Confirm & Pay [Authenticated — Customer Role]

← Confirm & Pay

Service Slot **3** Confirm

Booking Summary

Service Regular Wash

Slot Mon, Mar 3 · 08:00 – 10:00

Weight 5 kg

Estimated Total **₱ 225.00**

Final charge based on actual weight at shop.

Special Instructions (Optional)

e.g., separate whites from colors...

0 / 300

Secured by PayMongo

PROCEED TO PAYMENT

Overview

The final booking step presents a summary card of all selections, an optional special instructions field, and a payment trust badge. On confirming, the system locks the slot, creates the booking, and redirects to PayMongo checkout via a Custom Tab (Chrome Custom Tab) keeping the user within the app experience while loading the payment page.

Layout Structure

Top App Bar	56dp · bg #1E3A8A · back arrow · title 'Confirm & Pay'
Step Indicator	Step 3 active
Content	ScrollView · padding 16dp · single column
Sticky Bottom Bar	bg white · 'PROCEED TO PAYMENT' button full-width · PayMongo trust row above button

Components

Booking Summary Card

- White card · radius 12dp · elevation 1dp · padding 20dp
- Card header: 'Booking Summary' 16sp SemiBold #0F172A · booking ID will appear here after creation (shown as '—' before confirm)
- Detail rows (each 48dp tall with icon + label + value): Service — icon + name; Time Slot — calendar icon + date + time; Weight — scale icon + value + 'kg'; Total — money icon + '₪ X.XX' 18sp Bold #1D4ED8
- Thin 1dp #F1F5F9 dividers between each row
- Note below total: italic 12sp #94A3B8 'Final charge based on actual weight at shop.'

Special Instructions Input

- Section label: 'Special Instructions' 14sp SemiBold #334155 + '(Optional)' 12sp #94A3B8
- Multiline outlined input · minLines 3 · inputType textMultiLine · placeholder 'e.g., separate whites from colors...'
- Character counter: right-aligned 12sp #94A3B8 '0 / 300'

Sticky Bottom Bar

- Trust row: lock icon (20dp #16A34A) + 'Secured by PayMongo' 12sp #94A3B8 + PayMongo logo 60dp · centered · 8dp above button
- 'PROCEED TO PAYMENT' primary button · full-width · 48dp · bg #1D4ED8

States

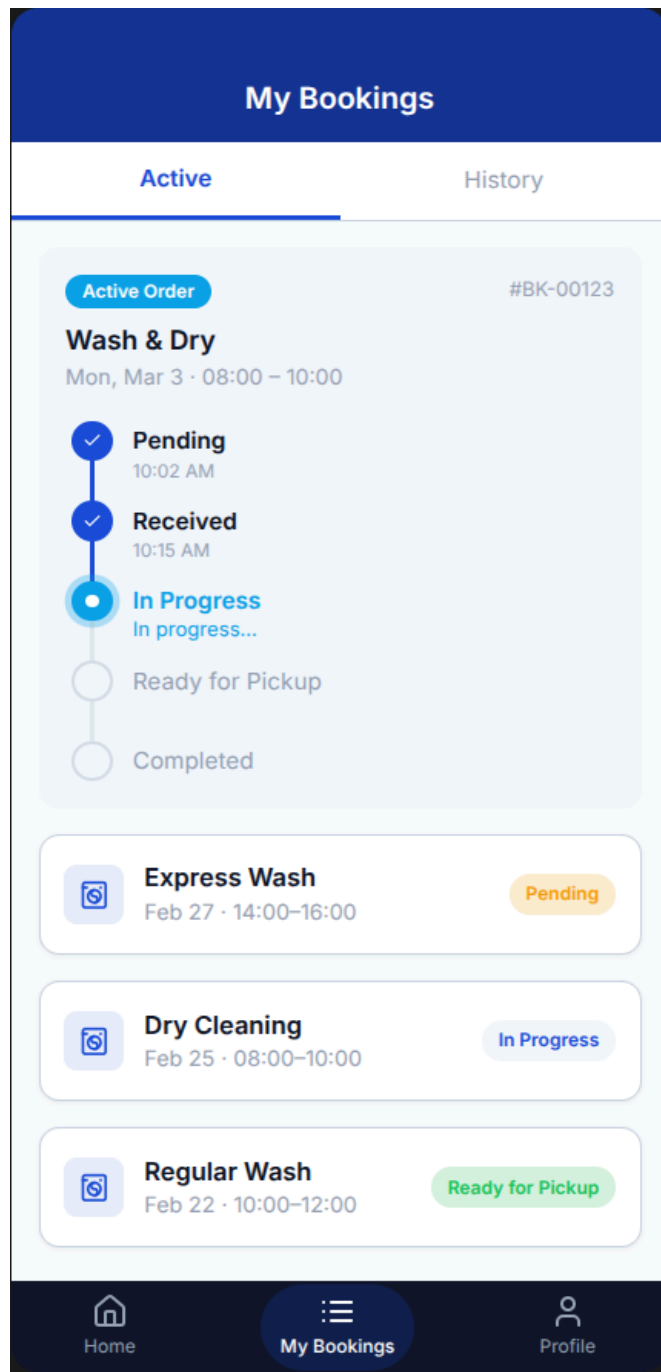
- Default — all details populated; special instructions empty; PROCEED button enabled
- Submitting — button shows CircularProgressIndicator (white, 20dp); all inputs disabled; a semi-transparent overlay on the summary card with a lock animation icon

- Slot conflict — bottom sheet appears: 'This slot was just filled.' title + body text + 'CHOOSE ANOTHER SLOT' primary button + 'CANCEL' text button
- Generic error — snackbar: 'Something went wrong. Please try again.' with 'RETRY' action
- Redirect to payment — Chrome Custom Tab opens PayMongo URL; app handles deep link callback on payment completion/failure

Micro-interactions & Android Patterns

- Character counter: text color changes to #D97706 at 250 chars and #DC2626 at 300 chars
- Lock animation on submitting: animated vector drawable — unlocked padlock transitions to locked state over 600ms
- Bottom sheet for slot conflict: slides up from bottom with rounded 16dp top corners; scrim dims content behind it
- Chrome Custom Tab: uses CustomTabsIntent.Builder with toolbar color #1E3A8A and close button at top-left so the experience feels native

Screen M-07.1 My Bookings / Order Status Screen - Active Tab [Authenticated — Customer Role]



Overview

Accessible from the bottom navigation 'Bookings' tab, this screen shows all of the user's bookings. The screen has two tabs: Active and History. The Active tab shows an order status tracker prominently for the most recent active booking, followed by a list of other active orders. The History tab shows past completed/cancelled orders.

Layout Structure

Top App Bar	56dp · bg #1E3A8A · title 'My Bookings' · no back arrow (root tab destination)
Tab Row	Below toolbar · 2 tabs: 'Active' and 'History' · bg white · indicator line #1D4ED8 · tab text 14sp SemiBold
Content	ViewPager2 with FragmentStateAdapter — swipe between Active and History tabs
Bottom Nav	Shared bottom navigation — 'My Bookings' tab active

Components

Active Tab — Order Status Tracker Card

- Appears only when there is at least one active booking · shows the most recent
- Card: bg #DBEAFE · radius 12dp · padding 16dp
- Header: 'Active Order' chip (#0EA5E9 filled) · booking ID 12sp right-aligned
- Service name 16sp SemiBold #0F172A · slot date + time 13sp #94A3B8 below
- Vertical status stepper (better suited to mobile than horizontal):
 - 5 steps stacked vertically, left-aligned
 - Each step: circle (24dp) on the left connected by vertical line · step label right + timestamp caption if reached
 - Completed step: filled #1D4ED8 circle + checkmark icon · label bold + timestamp
 - Current step: filled #0EA5E9 circle + pulsing ring animation · label bold #0EA5E9 · 'In progress...' caption
 - Upcoming steps: hollow circle #CBD5E1 · label #94A3B8
 - Vertical connector lines: 2dp · #CBD5E1 inactive, #1D4ED8 completed

Active Tab — Other Active Bookings List

- Section label: 'Other Active Bookings' 14sp SemiBold #94A3B8 · shown only if > 1 active booking
- Booking list cards: white · radius 12dp · elevation 1dp · padding 14dp · full-width
- Card layout: service icon 36dp left · service name 15sp SemiBold · slot date/time 13sp #94A3B8 · status badge pill right
- Swipe left on card reveals 'View Details' action (bg #1D4ED8, arrow icon) — ItemTouchHelper

Order Detail Bottom Sheet

- Full-height BottomSheetDialogFragment · bg white · top radius 16dp · drag handle
- Header: service name H2 + status badge + booking ID caption
- Detail rows: service, slot, weight, total amount, payment method, payment status badge
- Special instructions section (if present)
- 'CLOSE' text button bottom

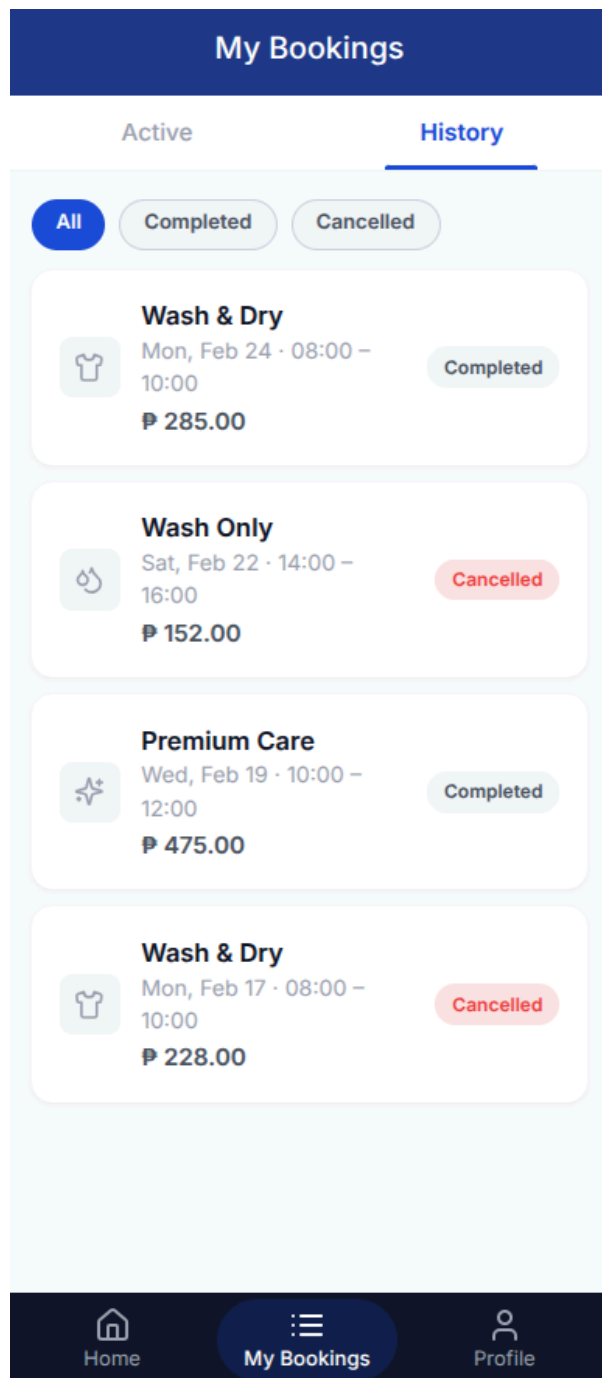
States

- Loading — tab content shows shimmer: 1 large shimmer block (tracker card) + 2 shimmer list items
- Active tab empty — no active bookings: illustration + 'No active orders.' + 'BOOK NOW' button navigating to Home
- History tab empty — 'No completed orders yet.' illustration + caption
- Pull-to-refresh — SwipeRefreshLayout on both tabs; spinner #1D4ED8

Micro-interactions & Android Patterns

- Current step pulse: ObjectAnimator on the ring stroke alpha and scale (1.0→1.5→1.0, 1500ms repeat indefinite)
- Tab switch: ViewPager2 page transition — smooth horizontal slide; tab indicator slides between tabs with spring interpolation
- Swipe-to-reveal: ItemTouchHelper with a custom draw callback paints the #1D4ED8 bg and arrow icon as the card slides left
- Bottom sheet expand: standard BottomSheetBehavior HALF_EXPANDED → EXPANDED with peek at 60% screen height

Screen M-07.2 My Bookings: History Tab [Authenticated — Customer Role]



Overview

The History tab is the second view within the My Bookings screen, accessible by tapping the "History" tab in the tab row. It shows all completed and cancelled bookings in reverse chronological order. A filter chip row allows the user to narrow results by status. This screen is a distinct visual state from M-07 (Active tab) and is reached via a ViewPager2 swipe or tab tap.

Layout Structure

Tab Row	RecyclerView, bg #F8FAFC, 16dp horizontal padding
Content	RecyclerView, bg #F8FAFC, 16dp horizontal padding
Filter Chip Row	HorizontalScrollView, chips for "All", "Completed", "Cancelled" — 8dp gap, padding 4dp vertical, 16dp top margin
Booking Card List	Stacked full-width cards, 10dp gap
Bottom Nav	Shared — "My Bookings" tab remains active

Components

Filter Chips

- "All" chip (default selected): bg #1D4ED8 · text white · 12sp SemiBold · radius 9999dp · padding 6dp 14dp
- "Completed" chip (unselected): bg #F1F5F9 · text #334155 · border #CBD5E1 · same radius and padding
- "Cancelled" chip (unselected): same as Completed chip style
- Only one chip active at a time (single-select); tapping filters the list below

Booking History Cards

- White card · radius 12dp · elevation 1dp · padding 16dp · full-width
- Layout: service icon 36dp #94A3B8 (intentionally muted compared to Active tab icons) left · text block (service name 15sp SemiBold #0F172A, slot date + time 13sp #94A3B8, amount paid 14sp SemiBold #334155) · status badge pill right
- Completed badge: bg #F1F5F9 · text #334155 · "Completed" · radius 9999dp · 11sp SemiBold
- Cancelled badge: bg #FEE2E2 · text #DC2626 · "Cancelled" · radius 9999dp · 11sp SemiBold
- Tapping a card open

Empty State

- Shown when no history exists or the active filter returns no results
- Centered layout: muted grey laundry basket or empty box illustration · "No past bookings yet." 16sp SemiBold #0F172A · "Your completed and cancelled orders will appear here." 13sp #94A3B8 · 24dp gap between illustration and text
- For filtered empty: "No [Completed / Cancelled] bookings." with "Clear filter" text button #1D4ED8 below

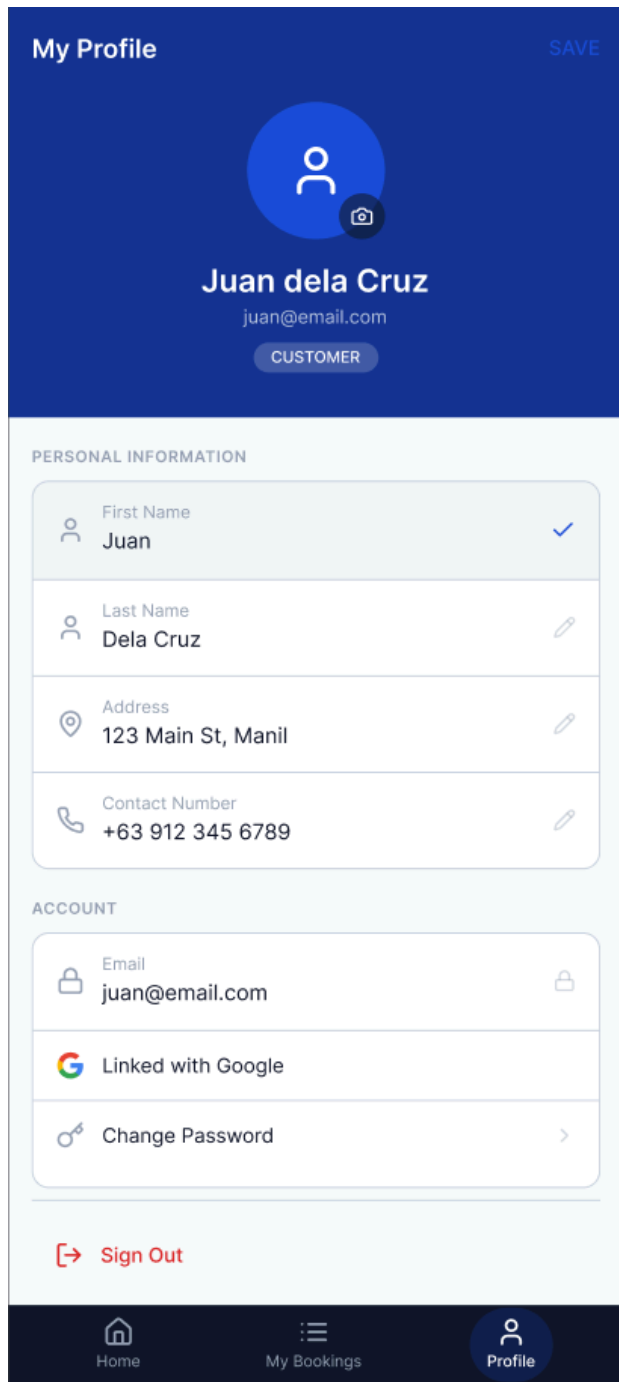
States

- Loading — shimmer cards (3 placeholders); filter chip row loads immediately
- Filtered empty — empty state with filter-specific message and "Clear filter" action
- Full empty — empty state with generic message; no CTA button needed
- Pull-to-refresh — SwipeRefreshLayout spinner #1D4ED8; refreshes the list

Micro-interactions & Android Patterns

- Tab switch from Active to History: ViewPager2 horizontal slide; tab indicator glides from "Active" to "History" position with spring interpolation
- Filter chip selection: bg and text color animate over 150ms; list fades in (alpha 0→1, 200ms) after filter applied to signal updated results
- Card tap: ripple effect on card surface before bottom sheet opens
- Empty state: illustration fades in with a gentle scale(0.9→1.0) + alpha(0→1) over 300ms on first appearance

Screen M-08 Profile Screen [Authenticated — Customer / Staff / Admin]



The image shows a mobile application profile screen. At the top, there's a dark blue header with 'My Profile' on the left and a 'SAVE' button on the right. Below the header is a circular profile picture placeholder with a person icon and a camera icon for uploading. Underneath the picture, the name 'Juan dela Cruz' is displayed, followed by the email 'juan@email.com' and a 'CUSTOMER' role badge. The main content area is divided into two sections: 'PERSONAL INFORMATION' and 'ACCOUNT'. The 'PERSONAL INFORMATION' section contains four rows: 'First Name' (Juan) with a checkmark, 'Last Name' (Dela Cruz) with an edit icon, 'Address' (123 Main St, Manil) with an edit icon, and 'Contact Number' (+63 912 345 6789) with an edit icon. The 'ACCOUNT' section contains three rows: 'Email' (juan@email.com) with a lock icon, 'Linked with Google' with the Google logo, and 'Change Password' with a key icon and a right arrow. At the bottom of the main content area is a red 'Sign Out' button with a right arrow. The bottom navigation bar has three items: 'Home' (house icon), 'My Bookings' (list icon), and 'Profile' (person icon, which is highlighted).

My Profile SAVE



Juan dela Cruz
juan@email.com
CUSTOMER

PERSONAL INFORMATION

 First Name
Juan ✓

 Last Name
Dela Cruz ✎

 Address
123 Main St, Manil ✎

 Contact Number
+63 912 345 6789 ✎

ACCOUNT

 Email
juan@email.com 🔒

 Linked with Google

 Change Password ➤

➡ Sign Out

Home My Bookings Profile

Overview

Accessible from the bottom navigation 'Profile' tab, this screen shows the user's account information, profile picture management, and sign-out. Editing is done inline in place — fields switch between display and edit mode on tap. Staff and Admin users see a role badge and have no booking-related actions.

Layout Structure

Top App Bar	56dp · bg #1E3A8A · title 'My Profile' · 'SAVE' text button right (appears only in edit mode)
Content	NestedScrollView · single column · no bottom padding beyond system inset
Bottom Nav	Shared bottom nav · 'Profile' tab active

Components

Profile Header Card

- Full-width card · bg #1E3A8A · padding 32dp 16dp · no radius (bleeds to top app bar for seamless look)
- Circular avatar: 96dp · circular clip · profile photo or initials fallback (white on #1D4ED8 bg circle)
- Camera overlay on avatar: semi-transparent dark circle overlay + camera icon 24dp white · appears on tap to trigger photo picker
- Name: 22sp SemiBold white below avatar · email: 13sp white/70%
- Role badge: 'CUSTOMER' / 'STAFF' / 'ADMIN' chip · bg white/20% · text white · 11sp · radius 9999dp

Personal Information Section

- Section label: 'PERSONAL INFORMATION' overline 11sp uppercase #94A3B8 · 16dp top margin
- Info rows (each a flat list item with divider): First Name, Last Name, Address, Contact Number
- Each row: icon left 24dp #94A3B8 · label 12sp #94A3B8 · value 15sp #0F172A · edit pencil icon right 20dp #CBD5E1
- Tapping a row: inline TextInputEditText replaces the value text; soft keyboard shows; row background turns #F1F5F9; pencil icon replaced by checkmark icon

Account Section

- Section label: 'ACCOUNT' overline · 20dp top margin
- Email row: lock icon left · label 'Email' · value (email address) · lock icon right (no edit; tapping shows tooltip 'Email cannot be changed')
- OAuth row (if applicable): Google icon 20dp · 'Linked with Google' 14sp #334155 · no edit
- 'Change Password' row: key icon left · label 14sp #334155 · chevron right — navigates to a dedicated Change Password screen

Sign Out Row

- 'SIGN OUT' — list item at bottom · log-out icon #DC2626 left · 'Sign Out' 14sp #DC2626 text · no chevron
- Tapping: confirmation dialog 'Sign out of WashQ?' with 'CANCEL' + 'SIGN OUT' (red) buttons

Photo Upload Flow

- Tapping avatar opens a ModalBottomSheet with two options: 'Take a photo' (camera icon) + 'Choose from gallery' (image icon)
- After selection: circular progress ring animates around the avatar (stroke #1D4ED8) during upload
- On success: new photo crossfades in; snackbar 'Profile photo updated.'
- On error: snackbar 'Upload failed. JPEG or PNG under 2 MB only.'

States

- View mode — all rows in display state; top app bar has no SAVE button
- Edit mode — at least one row in edit state; 'SAVE' appears in top app bar; keyboard visible
- Saving — SAVE replaced by circular progress in toolbar; edited rows dimmed
- Save success — rows return to display mode; snackbar 'Profile updated.'
- Save error — snackbar 'Update failed. Please try again.'

Micro-interactions & Android Patterns

- Inline edit transition: ValueAnimator changes row background color from white to #F1F5F9 (150ms); TextInputEditText fades in over display TextView
- Avatar tap: camera overlay fades in on ACTION_DOWN, fades out on ACTION_UP / ACTION_CANCEL (100ms)
- Progress ring: animated SVG circular indicator drawn programmatically around avatar circle
- Sign-out dialog: MaterialAlertDialog with custom destructive button style
- SAVE in toolbar: slides in from right (translateX 40dp→0, alpha 0→1, 200ms) when edit mode activates; slides back out on cancel or success

7.6 Mobile Screens Summary

The table below provides a quick reference of all eight mobile application screens.

ID	Screen Name	Access Role	Navigation Entry	Nav Chrome
M-01	Login Screen	Public	App launch / Sign out	None
M-02	Registration Screen	Public	From Login screen	Back arrow only
M-03	Home / Dashboard	Customer	Bottom Nav — Home tab	Top App Bar + Bottom Nav
M-04	Booking — Select Service	Customer	Home screen CTA or service card tap	Top App Bar (back)
M-05	Booking — Select Slot	Customer	Step 1 Continue	Top App Bar (back)
M-06	Booking — Confirm & Pay	Customer	Step 2 Continue	Top App Bar (back)
M-07.1	My Bookings / Order Status - Active Tab	Customer	Bottom Nav — Bookings tab → tap "Active"	Top App Bar + Bottom Nav
M-07.2	My Bookings — History Tab	Customer	Bottom Nav → Bookings tab → tap "History"	Top App Bar + Bottom Nav
M-08	Profile Screen	All roles	Bottom Nav — Profile tab	Top App Bar + Bottom Nav

8.0 PLAN

8.1 Project Timeline

The project is planned over 10 weeks. The schedule below is structured to prioritize backend stability before frontend development, and to front-load all mandatory integrations in Week 5 so they do not become last-minute risks.

Phase 1: Planning & Design (Week 1–2)

Week 1 – Requirements & Architecture:

- **Days 1–2:** Finalize project concept, gather requirements, and set up GitHub repository with the required structure
- **Days 3–4:** Complete functional and non-functional requirements in the SDD
- **Days 5–7:** Design system architecture and draw component diagram in draw.io

Week 2 – Detailed Design:

- **Days 1–2:** Define all API endpoint specifications and response structures
- **Days 3–4:** Design the database schema and create the ERD in draw.io
- **Days 5–6:** Create UI/UX wireframes in Figma for web and mobile
- **Day 7:** Finalize SDD and implementation plan

Phase 2: Backend Development (Week 3–4)

Week 3 – Foundation:

- **Day 1:** Spring Boot project setup (dependencies, application.properties, PostgreSQL/Supabase connection)
- **Day 2:** JPA entities and repositories (User, Service, TimeSlot, Booking, Payment)
- **Day 3:** JWT authentication implementation (Spring Security, BCrypt, JwtFilter)
- **Day 4:** Auth endpoints (register, login, logout, /me)
- **Day 5:** Service CRUD endpoints

Week 4 – Core Business Module:

- **Day 1:** Time slot management endpoints and availability logic

- **Day 2:** Booking creation with transactional slot locking and double-booking prevention
- **Day 3:** Booking status update workflow and retrieval endpoints
- **Day 4:** Global exception handling, input validation, and DTO refinement
- **Day 5:** Backend unit and integration testing (JUnit/MockMvc)

Phase 3: Backend Integrations (Week 5)

All mandatory integrations are implemented in a dedicated week to avoid last-minute surprises.

- **Day 1:** Google OAuth 2.0 integration — Spring Security OAuth2, JWT issuance after OAuth
- **Day 2:** SMTP email integration — JavaMailSender, welcome email and order-ready notification templates
- **Day 3:** PayMongo integration — checkout session creation, webhook receiver, payment status update
- **Day 4:** OpenWeatherMap integration — backend proxy endpoint, API key management
- **Day 5:** File upload integration — Spring Boot MultipartFile, profile picture storage, user record linking

Phase 4: Web Frontend (Week 6–7)

Week 6 – Foundation:

- **Day 1:** React project setup (Vite, Tailwind CSS, Axios, React Router)
- **Day 2:** Authentication pages (Registration, Login, Google OAuth button)
- **Day 3:** JWT handling, protected routes (PrivateRoute), Axios interceptor
- **Day 4:** Customer Dashboard (service catalog, weather widget, current bookings overview)
- **Day 5:** Booking flow (service selection, slot selection, booking confirmation)

Week 7 – Complete Web Features:

- **Day 1:** Payment redirect flow integration with PayMongo
- **Day 2:** Order history and order status tracking page
- **Day 3:** Staff/Admin dashboard — booking management, status updates
- **Day 4:** Admin service management (add/edit/deactivate services)
- **Day 5:** Profile page with profile picture upload; responsive design polish

Phase 5: Mobile Application (Week 8)

- **Day 1:** Android Studio project setup (Kotlin, Gradle, Retrofit 2, OkHttp)
- **Day 2:** Login and Registration screens with JWT storage in EncryptedSharedPreferences
- **Day 3:** Home/Dashboard screen, Service List screen, and weather widget
- **Day 4:** Booking flow screens (slot selection, confirmation); role-aware navigation
- **Day 5:** Order Status screen, Profile screen; error handling and UI polish

Phase 6: Integration Testing & Deployment (Week 9–10)

Week 9 – Integration & Testing:

- **Day 1–2:** End-to-end testing across all three platforms (web, mobile, backend)
- **Day 3:** Security review (JWT handling, protected routes, OAuth flow, webhook verification)
- **Day 4:** Bug fixes, performance optimization, and API edge case handling
- **Day 5:** Final SDD update — insert diagrams, wireframes, and document all changes

Week 10 – Final Polish & Submission:

- **Days 1–2:** Backend deployment and web application deployment
- **Day 3:** Android APK generation and testing on physical device or emulator
- **Day 4:** Final end-to-end smoke test across all platforms
- **Day 5:** Final SDD submission and project defense preparation

8.2 Milestones

Milestone	Target Date	Deliverable
M1	End of Week 2	All design documents and SDD complete
M2	End of Week 4	Backend API fully functional with auth, services, and bookings
M3	End of Week 5	All backend integrations complete (PayMongo, SMTP, OAuth, File Upload, OpenWeatherMap)
M4	End of Week 7	Web application (React) fully functional
M5	End of Week 8	Android mobile application complete

M6	End of Week 10	Full system integrated, tested, deployed, and submitted
-----------	----------------	---

8.3 Risk Mitigation

- Start with the simplest working version of each feature before adding complexity.
- All mandatory integrations (PayMongo, OAuth, SMTP, File Upload, OpenWeatherMap) are scheduled together in Week 5 so issues are discovered early.
- Test backend API endpoints with Postman immediately after each is implemented — do not batch all testing to Week 9.
- Keep a working backup branch for each milestone phase.
- If PayMongo integration proves more complex than expected, refer to the official PayMongo API documentation and use the PayMongo Postman collection for sandbox testing.
- Android development is constrained by the XML layout requirement (Jetpack Compose is not allowed). Plan to follow traditional Activity/Fragment patterns using ViewBinding.